ADS Mini Project

```
# ============================================
# ADS MINI PROJECT
# Retail Sales Intelligence
# ============================================

# Data manipulation
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Display settings
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
from google.colab import files

uploaded = files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
# Read the Excel file
file_name = "sample_-_superstore.xlsx"

excel_file = pd.ExcelFile(file_name)

# Display available sheets
print("Available sheets:")
print(excel_file.sheet_names)
```

```
Available sheets:
['Orders', 'People', 'Returns']
```

```
# Load Orders sheet
df = pd.read_excel(file_name, sheet_name='Orders')

print("Orders sheet loaded successfully!")
print("Dataset shape:", df.shape)
```

```
Orders sheet loaded successfully!
Dataset shape: (10194, 21)
```

```
display(df.head())
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country/Region	City	State/Province	Postal Code	Regi
0	1	US-2023-103800	2023-01-03	2023-01-07	Standard Class	DP-13000	Darren Powers	Consumer	United States	Houston	Texas	77095	Cen
1	2	US-2023-112326	2023-01-04	2023-01-08	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Naperville	Illinois	60540	Cen
2	3	US-2023-112326	2023-01-04	2023-01-08	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Naperville	Illinois	60540	Cen
3	4	US-2023-112326	2023-01-04	2023-01-08	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Naperville	Illinois	60540	Cen
4	5	US-2023-141817	2023-01-05	2023-01-12	Standard Class	MB-18085	Mick Brown	Consumer	United States	Philadelphia	Pennsylvania	19143	E

```
display(df.tail())
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country/Region	City	State/Province	Post Co
10189	10190	US-2026-143259	2026-12-30	2027-01-03	Standard Class	PO-18865	Patrick O'Donnell	Consumer	United States	New York City	New York	100
10190	10191	US-2026-115427	2026-12-30	2027-01-03	Standard Class	EB-13975	Erica Bern	Corporate	United States	Fairfield	California	945
10191	10192	US-2026-156720	2026-12-30	2027-01-03	Standard Class	JM-15580	Jill Matthias	Consumer	United States	Loveland	Colorado	805
10192	10193	US-2026-143259	2026-12-30	2027-01-03	Standard Class	PO-18865	Patrick O'Donnell	Consumer	United States	New York City	New York	100
10193	10194	CA-2026-143500	2026-12-30	2027-01-03	Standard Class	HO-15230	Harry Olson	Consumer	Canada	Charlottetown	Prince Edward Island	C

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

```
Number of rows: 10194
Number of columns: 21
```

```
print("Column Names:\n")

for i, column in enumerate(df.columns, start=1):
    print(f"{i}. {column}")
```

```
Column Names:

1. Row ID
2. Order ID
3. Order Date
4. Ship Date
5. Ship Mode
6. Customer ID
7. Customer Name
8. Segment
9. Country/Region
```

```
10. City
11. State/Province
12. Postal Code
13. Region
14. Product ID
15. Category
16. Sub-Category
17. Product Name
18. Sales
19. Quantity
20. Discount
21. Profit
```

```
print("Dataset Information:\n")

df.info()
```

```
Dataset Information:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10194 entries, 0 to 10193
Data columns (total 21 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Row ID          10194 non-null  int64
 1   Order ID        10194 non-null  object
 2   Order Date      10194 non-null  datetime64[ns]
 3   Ship Date       10194 non-null  datetime64[ns]
 4   Ship Mode       10194 non-null  object
 5   Customer ID     10194 non-null  object
 6   Customer Name   10194 non-null  object
 7   Segment         10194 non-null  object
 8   Country/Region  10194 non-null  object
 9   City            10194 non-null  object
 10  State/Province  10194 non-null  object
 11  Postal Code     10194 non-null  object
 12  Region          10194 non-null  object
 13  Product ID      10194 non-null  object
 14  Category        10194 non-null  object
 15  Sub-Category    10194 non-null  object
 16  Product Name    10194 non-null  object
 17  Sales           10194 non-null  float64
 18  Quantity        10194 non-null  int64
 19  Discount        10194 non-null  float64
 20  Profit          10194 non-null  float64
dtypes: datetime64[ns](2), float64(3), int64(2), object(14)
memory usage: 1.6+ MB
```

```
dtype_table = pd.DataFrame({
    'Column': df.columns,
    'Data Type': df.dtypes.astype(str).values
})

display(dtype_table)
```

	Column	Data Type
0	Row ID	int64
1	Order ID	object
2	Order Date	datetime64[ns]
3	Ship Date	datetime64[ns]
4	Ship Mode	object
5	Customer ID	object
6	Customer Name	object
7	Segment	object
8	Country/Region	object
9	City	object
10	State/Province	object
11	Postal Code	object
12	Region	object
13	Product ID	object
14	Category	object
15	Sub-Category	object
16	Product Name	object
17	Sales	float64
18	Quantity	int64
19	Discount	float64
20	Profit	float64

```
missing = pd.DataFrame({
    'Column': df.columns,
    'Missing Values': df.isnull().sum(),
    'Missing Percentage': (df.isnull().sum() / len(df) * 100).round(2)
})

display(missing)
```

	Column	Missing Values	Missing Percentage
Row ID	Row ID	0	0.0
Order ID	Order ID	0	0.0
Order Date	Order Date	0	0.0
Ship Date	Ship Date	0	0.0
Ship Mode	Ship Mode	0	0.0
Customer ID	Customer ID	0	0.0
Customer Name	Customer Name	0	0.0
Segment	Segment	0	0.0
Country/Region	Country/Region	0	0.0
City	City	0	0.0
State/Province	State/Province	0	0.0
Postal Code	Postal Code	0	0.0
Region	Region	0	0.0
Product ID	Product ID	0	0.0
Category	Category	0	0.0
Sub-Category	Sub-Category	0	0.0
Product Name	Product Name	0	0.0
Sales	Sales	0	0.0
Quantity	Quantity	0	0.0
Discount	Discount	0	0.0
Profit	Profit	0	0.0

```
duplicate_count = df.duplicated().sum()

print("Number of duplicate records:", duplicate_count)
print("Percentage of duplicate records:",
      round((duplicate_count / len(df)) * 100, 2), "%")
```

```
Number of duplicate records: 0
Percentage of duplicate records: 0.0 %
```

```
numeric_summary = df.describe()

display(numeric_summary)
```

	Row ID	Order Date	Ship Date	Sales	Quantity	Discount	Profit
count	10194.000000	10194	10194	10194.000000	10194.000000	10194.000000	10194.000000
mean	5097.500000	2025-04-29 11:48:25.002942720	2025-05-03 10:52:45.626839296	228.225854	3.791838	0.155385	28.673417
min	1.000000	2023-01-03 00:00:00	2023-01-07 00:00:00	0.444000	1.000000	0.000000	-6599.978000
25%	2549.250000	2024-05-14 00:00:00	2024-05-19 00:00:00	17.220000	2.000000	0.000000	1.760800
50%	5097.500000	2025-06-25 00:00:00	2025-06-28 00:00:00	53.910000	3.000000	0.200000	8.690000
75%	7645.750000	2026-05-14 00:00:00	2026-05-18 00:00:00	209.500000	5.000000	0.200000	29.297925
max	10194.000000	2026-12-30 00:00:00	2027-01-05 00:00:00	22638.480000	14.000000	0.800000	8399.976000

```
numeric_columns = df.select_dtypes(include=np.number).columns.tolist()

print("Numerical columns:")
print(numeric_columns)
```

```
Numerical columns:
['Row ID', 'Sales', 'Quantity', 'Discount', 'Profit']
```

```
categorical_columns = df.select_dtypes(
    include=['object', 'category']
).columns.tolist()

print("Categorical columns:")
print(categorical_columns)
```

```
Categorical columns:
['Order ID', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Country/Region', 'City', 'State/Province', 'Postal Coc
```

```
unique_values = pd.DataFrame({
    'Column': df.columns,
    'Unique Values': [df[col].nunique() for col in df.columns]
})

display(unique_values)
```

	Column	Unique Values
0	Row ID	10194
1	Order ID	5111
2	Order Date	1242
3	Ship Date	1338
4	Ship Mode	4
5	Customer ID	804
6	Customer Name	800
7	Segment	3
8	Country/Region	2
9	City	542
10	State/Province	59
11	Postal Code	654
12	Region	4
13	Product ID	1862
14	Category	3
15	Sub-Category	17
16	Product Name	1849
17	Sales	6161
18	Quantity	14
19	Discount	12
20	Profit	7597

```
unique_values = pd.DataFrame({
    'Column': df.columns,
    'Unique Values': [df[col].nunique() for col in df.columns]
})

display(unique_values)
```

	Column	Unique Values
0	Row ID	10194
1	Order ID	5111
2	Order Date	1242
3	Ship Date	1338
4	Ship Mode	4
5	Customer ID	804
6	Customer Name	800
7	Segment	3
8	Country/Region	2
9	City	542
10	State/Province	59
11	Postal Code	654
12	Region	4
13	Product ID	1862
14	Category	3
15	Sub-Category	17
16	Product Name	1849
17	Sales	6161
18	Quantity	14
19	Discount	12
20	Profit	7597

```
print("Order Date type:", df['Order Date'].dtype)
print("Ship Date type:", df['Ship Date'].dtype)

print("\nOrder Date range:")
print("Minimum:", df['Order Date'].min())
print("Maximum:", df['Order Date'].max())
```

```
Order Date type: datetime64[ns]
Ship Date type: datetime64[ns]

Order Date range:
Minimum: 2023-01-03 00:00:00
Maximum: 2026-12-30 00:00:00
```

```
important_columns = [
    'Sales',
    'Quantity',
    'Discount',
    'Profit'
]

display(df[important_columns].describe().T)
```

	count	mean	std	min	25%	50%	75%	max
Sales	10194.0	228.225854	619.906839	0.444	17.2200	53.91	209.500000	22638.480
Quantity	10194.0	3.791838	2.228317	1.000	2.0000	3.00	5.000000	14.000
Discount	10194.0	0.155385	0.206249	0.000	0.0000	0.20	0.200000	0.800
Profit	10194.0	28.673417	232.465115	-6599.978	1.7608	8.69	29.297925	8399.976

```
print("Categories:")
print(df['Category'].unique())

print("\nSub-Categories:")
print(df['Sub-Category'].unique())

print("\nRegions:")
print(df['Region'].unique())

print("\nSegments:")
print(df['Segment'].unique())
```

```
Categories:
['Office Supplies' 'Furniture' 'Technology']

Sub-Categories:
['Paper' 'Binders' 'Labels' 'Storage' 'Art' 'Chairs' 'Fasteners' 'Phones'
 'Furnishings' 'Accessories' 'Bookcases' 'Envelopes' 'Appliances' 'Tables'
 'Supplies' 'Machines' 'Copiers']

Regions:
['Central' 'East' 'South' 'West']

Segments:
['Consumer' 'Home Office' 'Corporate']
```

```
print("Category distribution:")
display(df['Category'].value_counts())
```

Category distribution:

Category	count
Office Supplies	6128
Furniture	2201
Technology	1865

dtype: int64

```
print("Region distribution:")
display(df['Region'].value_counts())
```

Region distribution:

Region	count
West	3253
East	2986
Central	2335
South	1620

dtype: int64

```
print("Segment distribution:")
display(df['Segment'].value_counts())
```

Segment distribution:

Segment	count
Consumer	5281
Corporate	3090
Home Office	1823

dtype: int64

```
for col in ['Sales', 'Quantity', 'Discount', 'Profit']:
    print(f"\n{col}")
    print("Minimum:", df[col].min())
    print("Maximum:", df[col].max())
    print("Zero values:", (df[col] == 0).sum())
    print("Negative values:", (df[col] < 0).sum())
```

```
Sales
Minimum: 0.44399999999999995
Maximum: 22638.48
Zero values: 0
Negative values: 0

Quantity
Minimum: 1
Maximum: 14
Zero values: 0
Negative values: 0

Discount
Minimum: 0.0
Maximum: 0.8
Zero values: 4925
Negative values: 0

Profit
Minimum: -6599.978000000001
Maximum: 8399.975999999999
Zero values: 65
Negative values: 1901
```

```
quality_report = pd.DataFrame({
    'Column': df.columns,
    'Data Type': df.dtypes.astype(str).values,
    'Missing Values': df.isnull().sum().values,
    'Unique Values': [df[col].nunique() for col in df.columns]
})

quality_report['Missing %'] = (
    quality_report['Missing Values'] / len(df) * 100
).round(2)

display(quality_report)
```

	Column	Data Type	Missing Values	Unique Values	Missing %
0	Row ID	int64	0	10194	0.0
1	Order ID	object	0	5111	0.0
2	Order Date	datetime64[ns]	0	1242	0.0
3	Ship Date	datetime64[ns]	0	1338	0.0
4	Ship Mode	object	0	4	0.0
5	Customer ID	object	0	804	0.0
6	Customer Name	object	0	800	0.0
7	Segment	object	0	3	0.0
8	Country/Region	object	0	2	0.0
9	City	object	0	542	0.0
10	State/Province	object	0	59	0.0
11	Postal Code	object	0	654	0.0
12	Region	object	0	4	0.0
13	Product ID	object	0	1862	0.0
14	Category	object	0	3	0.0
15	Sub-Category	object	0	17	0.0
16	Product Name	object	0	1849	0.0
17	Sales	float64	0	6161	0.0
18	Quantity	int64	0	14	0.0
19	Discount	float64	0	12	0.0
20	Profit	float64	0	7597	0.0

```
print("=" * 60)
print("DATASET SUMMARY")
print("=" * 60)

print(f"Number of records       : {df.shape[0]}")
print(f"Number of features      : {df.shape[1]}")
print(f"Missing values          : {df.isnull().sum().sum()}")
```

```
print(f"Duplicate records       : {df.duplicated().sum()}")
print(f"Numerical features      : {len(numeric_columns)}")
print(f"Categorical features    : {len(categorical_columns)}")
print(f"Order date range        : {df['Order Date'].min()} to {df['Order Date'].max()}")

print("=" * 60)
```

```
============================================================
DATASET SUMMARY
============================================================
Number of records       : 10194
Number of features      : 21
Missing values          : 0
Duplicate records       : 0
Numerical features      : 5
Categorical features    : 14
Order date range        : 2023-01-03 00:00:00 to 2026-12-30 00:00:00
============================================================
```

```
df_raw = df.copy()

print("Raw dataset backup created successfully.")
```

```
Raw dataset backup created successfully.
```

Data Cleaning and Feature Extraction

```
# Create working copy
df = df_raw.copy()

print("Working dataset created.")
print("Shape:", df.shape)
```

```
Working dataset created.
Shape: (10194, 21)
```

```
# Convert date columns explicitly to datetime
df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Ship Date'] = pd.to_datetime(df['Ship Date'])

print("Order Date type:", df['Order Date'].dtype)
print("Ship Date type:", df['Ship Date'].dtype)
```

```
Order Date type: datetime64[ns]
Ship Date type: datetime64[ns]
```

```
missing_total = df.isnull().sum().sum()

print("Total missing values:", missing_total)

if missing_total == 0:
    print("✓ Dataset contains no missing values.")
else:
    print("⚠ Missing values detected.")
```

```
Total missing values: 0
✓ Dataset contains no missing values.
```

```
missing_total = df.isnull().sum().sum()

print("Total missing values:", missing_total)

if missing_total == 0:
    print("✓ Dataset contains no missing values.")
else:
    print("⚠ Missing values detected.")
```

```
Total missing values: 0
✓ Dataset contains no missing values.
```

```
duplicate_total = df.duplicated().sum()

print("Total duplicate records:", duplicate_total)

if duplicate_total == 0:
    print("✓ Dataset contains no duplicate records.")
else:
    print("⚠ Duplicate records detected.")
```

```
Total duplicate records: 0
✓ Dataset contains no duplicate records.
```

```
numeric_columns = df.select_dtypes(include=np.number).columns.tolist()

print("Numerical columns:")
for col in numeric_columns:
    print("-", col)
```

```
Numerical columns:
- Row ID
- Sales
- Quantity
- Discount
- Profit
```

```
# Check for infinite values
infinite_values = np.isinf(
    df[numeric_columns]
).sum().sum()

print("Total infinite values:", infinite_values)
```

```
Total infinite values: 0
```

```
# Date-based feature engineering

df['Year'] = df['Order Date'].dt.year
df['Month'] = df['Order Date'].dt.month
df['Month Name'] = df['Order Date'].dt.month_name()
df['Quarter'] = df['Order Date'].dt.quarter
df['Day'] = df['Order Date'].dt.day
df['Day of Week'] = df['Order Date'].dt.day_name()
df['Week'] = df['Order Date'].dt.isocalendar().week.astype(int)

display(
    df[
        [
            'Order Date',
            'Year',
            'Month',
            'Month Name',
            'Quarter',
            'Day',
            'Day of Week',
            'Week'
        ]
    ].head()
)
```

	Order Date	Year	Month	Month Name	Quarter	Day	Day of Week	Week
0	2023-01-03	2023	1	January	1	3	Tuesday	1
1	2023-01-04	2023	1	January	1	4	Wednesday	1
2	2023-01-04	2023	1	January	1	4	Wednesday	1
3	2023-01-04	2023	1	January	1	4	Wednesday	1
4	2023-01-05	2023	1	January	1	5	Thursday	1

```
# Profit Margin (%)

df['Profit Margin'] = np.where(
    df['Sales'] != 0,
    (df['Profit'] / df['Sales']) * 100,
    0
)

display(
    df[
        ['Sales', 'Profit', 'Profit Margin']
    ].head()
)
```

	Sales	Profit	Profit Margin
0	16.448	5.5512	33.75
1	3.540	-5.4870	-155.00
2	11.784	4.2717	36.25
3	272.736	-64.7748	-23.75
4	19.536	4.8840	25.00

```
# Calculate shipping duration

df['Shipping Days'] = (
    df['Ship Date'] - df['Order Date']
).dt.days

display(
    df[
        ['Order Date', 'Ship Date', 'Shipping Days']
    ].head()
)
```

	Order Date	Ship Date	Shipping Days
0	2023-01-03	2023-01-07	4
1	2023-01-04	2023-01-08	4
2	2023-01-04	2023-01-08	4
3	2023-01-04	2023-01-08	4
4	2023-01-05	2023-01-12	7

```
print("Shipping Days statistics:")
display(df['Shipping Days'].describe())
```

```
Shipping Days statistics:
```

	Shipping Days
count	10194.000000
mean	3.961350
std	1.742829
min	0.000000
25%	3.000000
50%	4.000000
75%	5.000000
max	11.000000

dtype: float64

```
negative_shipping = (df['Shipping Days'] < 0).sum()

print("Negative shipping durations:", negative_shipping)
```

```
Negative shipping durations: 0
```

```
df['Order Month'] = df['Order Date'].dt.to_period('M')

display(
    df[
        ['Order Date', 'Order Month', 'Sales']
    ].head()
)
```

	Order Date	Order Month	Sales
0	2023-01-03	2023-01	16.448
1	2023-01-04	2023-01	3.540
2	2023-01-04	2023-01	11.784
3	2023-01-04	2023-01	272.736
4	2023-01-05	2023-01	19.536

```
print("Original number of columns:", df_raw.shape[1])
print("New number of columns:", df.shape[1])

print("\nNew columns created:")

new_columns = [
    col for col in df.columns
    if col not in df_raw.columns
]

for col in new_columns:
    print("-", col)
```

```
Original number of columns: 21
New number of columns: 31

New columns created:
- Year
- Month
- Month Name
- Quarter
- Day
- Day of Week
- Week
- Profit Margin
- Shipping Days
- Order Month
```

```
final_quality = pd.DataFrame({
    'Metric': [
        'Rows',
        'Columns',
        'Missing Values',
        'Duplicate Records',
        'Infinite Values'
    ],
    'Value': [
        df.shape[0],
        df.shape[1],
        df.isnull().sum().sum(),
        df.duplicated().sum(),
        np.isinf(
            df.select_dtypes(include=np.number)
        ).sum().sum()
    ]
})

display(final_quality)
```

	Metric	Value
0	Rows	10194
1	Columns	31
2	Missing Values	0
3	Duplicate Records	0
4	Infinite Values	0

```
# Save processed dataset
processed_file = 'superstore_processed.csv'

df.to_csv(
    processed_file,
    index=False
)

print(f"Processed dataset saved as: {processed_file}")
```

```
Processed dataset saved as: superstore_processed.csv
```

```
from google.colab import files

files.download(processed_file)
```

PART 3 — Exploratory Data Analysis (EDA)

```
print("Dataset shape:", df.shape)

display(df.head())
```

```
Dataset shape: (10194, 31)
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country/Region	City	State/Province	Postal Code	Regi
0	1	US-2023-103800	2023-01-03	2023-01-07	Standard Class	DP-13000	Darren Powers	Consumer	United States	Houston	Texas	77095	Cen
1	2	US-2023-112326	2023-01-04	2023-01-08	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Naperville	Illinois	60540	Cen
2	3	US-2023-112326	2023-01-04	2023-01-08	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Naperville	Illinois	60540	Cen
3	4	US-2023-112326	2023-01-04	2023-01-08	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Naperville	Illinois	60540	Cen
4	5	US-2023-141817	2023-01-05	2023-01-12	Standard Class	MB-18085	Mick Brown	Consumer	United States	Philadelphia	Pennsylvania	19143	E

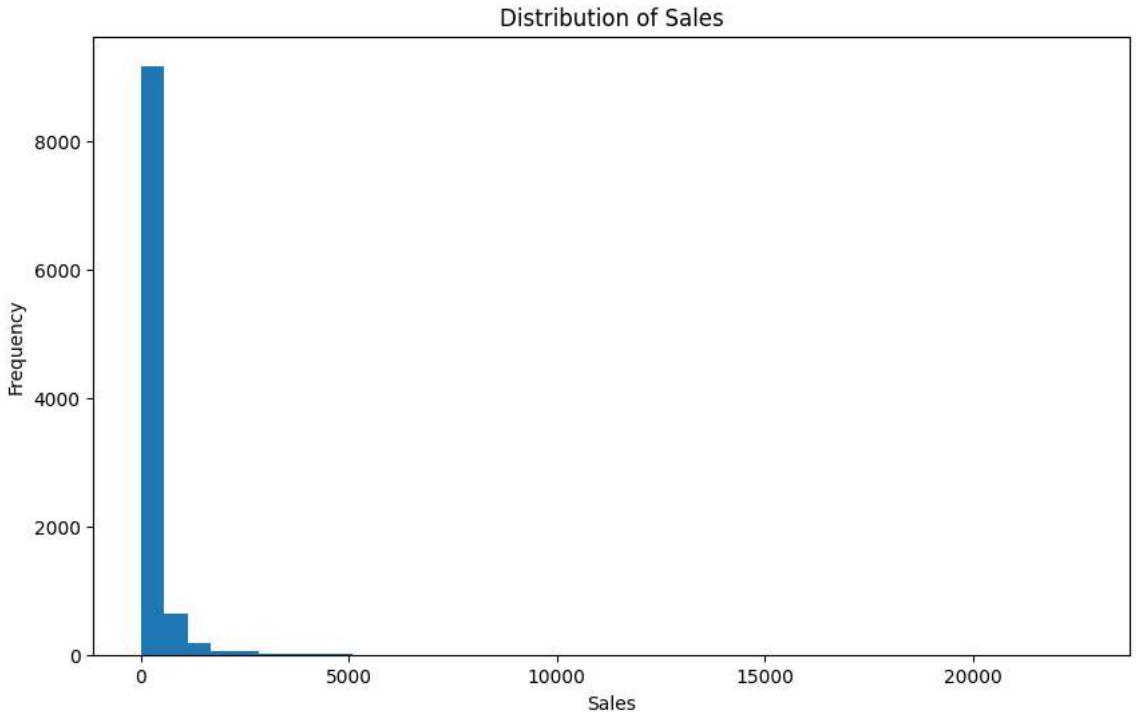
Univarite Analysis

```
#sales distribution
plt.figure(figsize=(10, 6))

plt.hist(df['Sales'], bins=40)

plt.title('Distribution of Sales')
plt.xlabel('Sales')
plt.ylabel('Frequency')

plt.show()
```

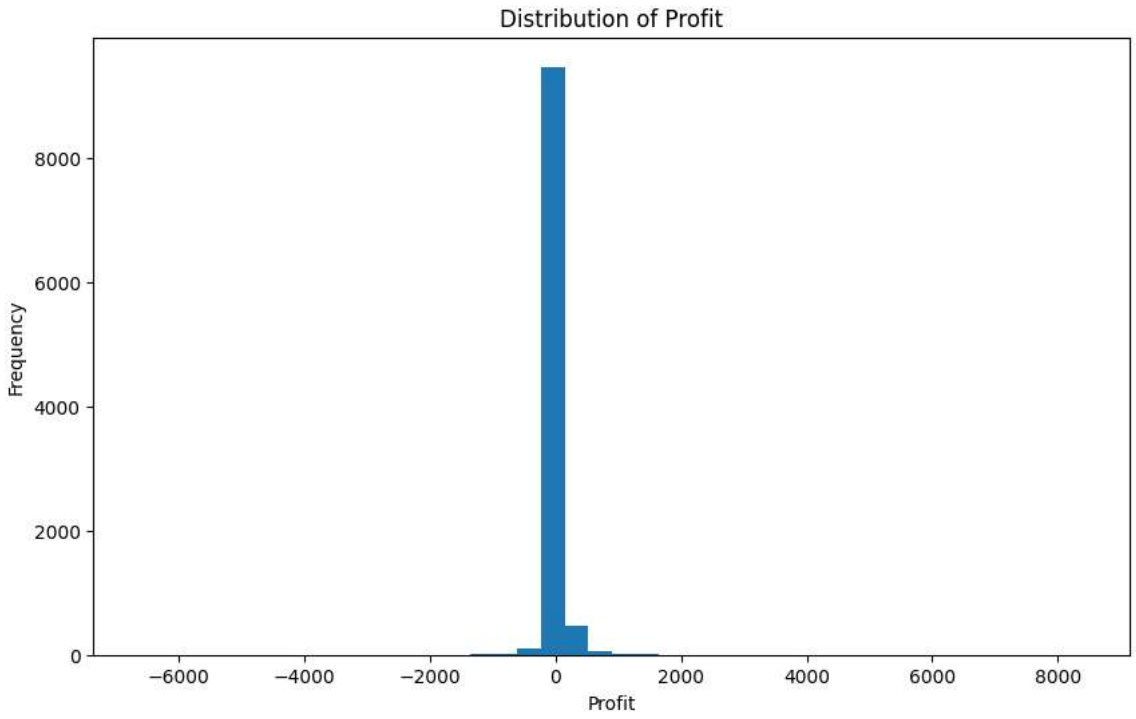



```
#profit distribution
plt.figure(figsize=(10, 6))

plt.hist(df['Profit'], bins=40)

plt.title('Distribution of Profit')
plt.xlabel('Profit')
plt.ylabel('Frequency')

plt.show()
```

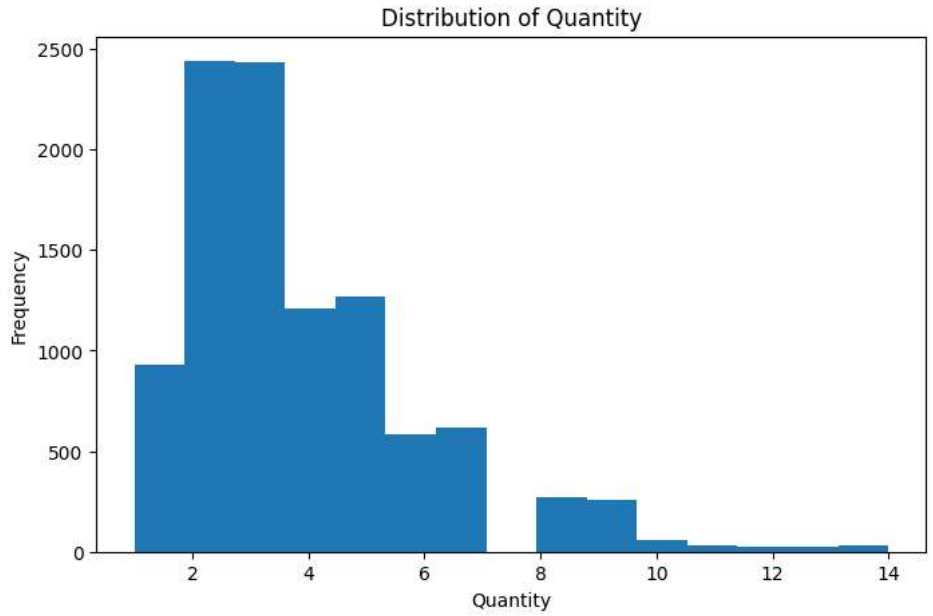


```
#quantity distibution
plt.figure(figsize=(8, 5))

plt.hist(df['Quantity'], bins=15)

plt.title('Distribution of Quantity')
plt.xlabel('Quantity')
plt.ylabel('Frequency')

plt.show()
```

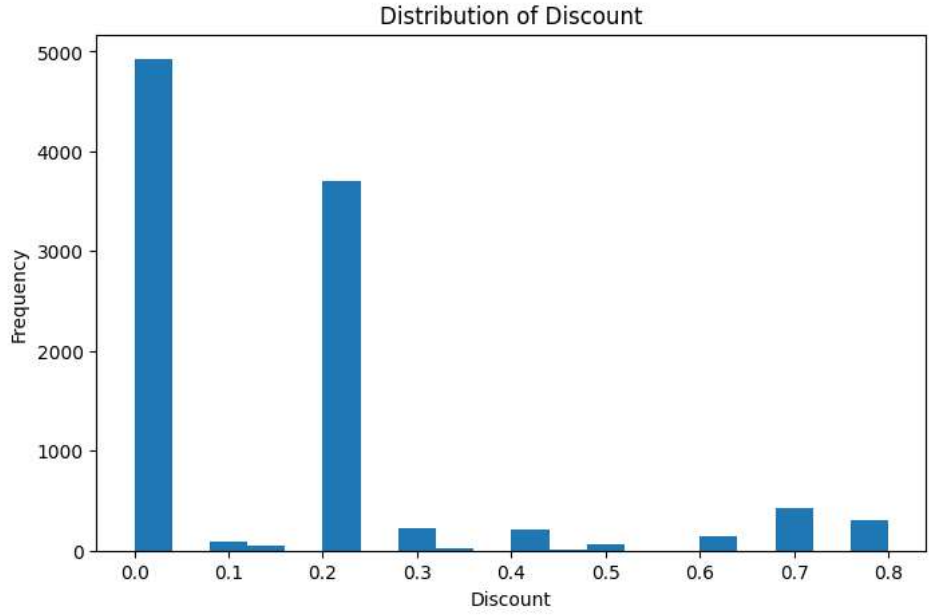



```
#discount distribution
plt.figure(figsize=(8, 5))

plt.hist(df['Discount'], bins=20)

plt.title('Distribution of Discount')
plt.xlabel('Discount')
plt.ylabel('Frequency')

plt.show()
```

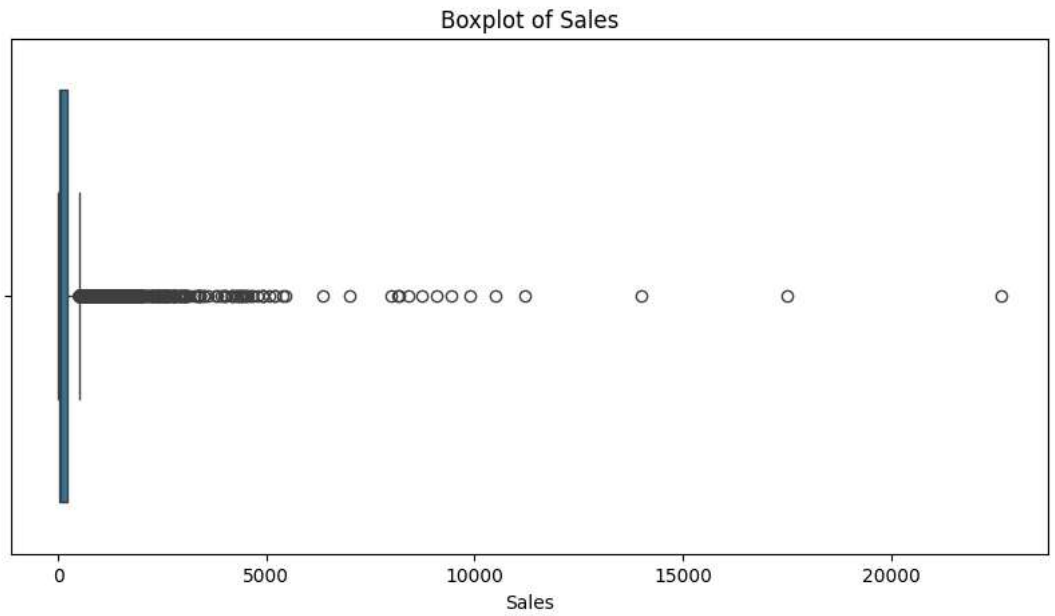



```
#sales boxplot
plt.figure(figsize=(10, 5))

sns.boxplot(x=df['Sales'])

plt.title('Boxplot of Sales')
plt.xlabel('Sales')
```

```
plt.show()
```

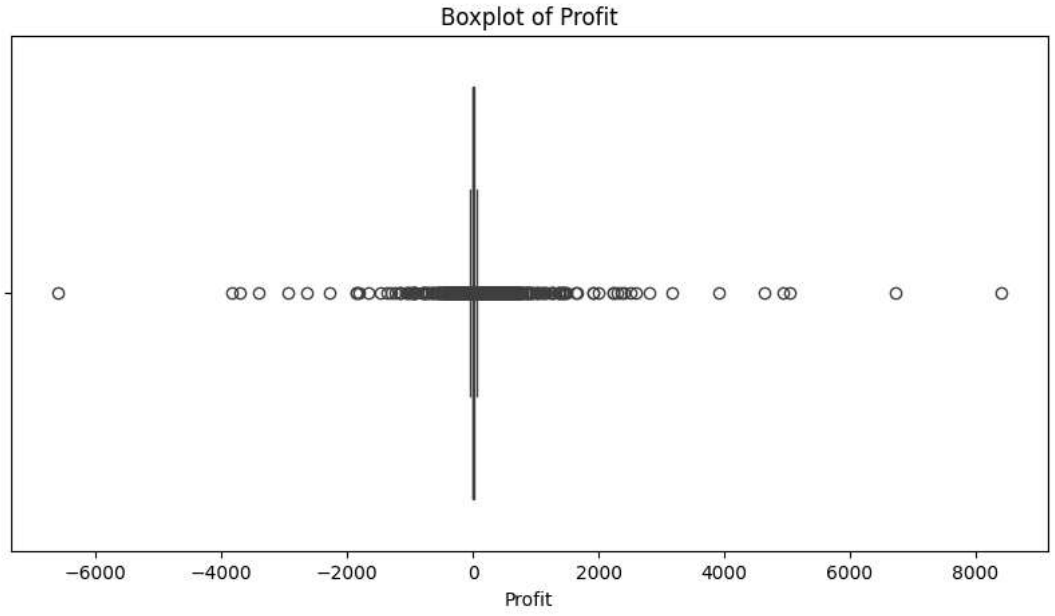



```
#profit boxplot
plt.figure(figsize=(10, 5))

sns.boxplot(x=df['Profit'])

plt.title('Boxplot of Profit')
plt.xlabel('Profit')

plt.show()
```

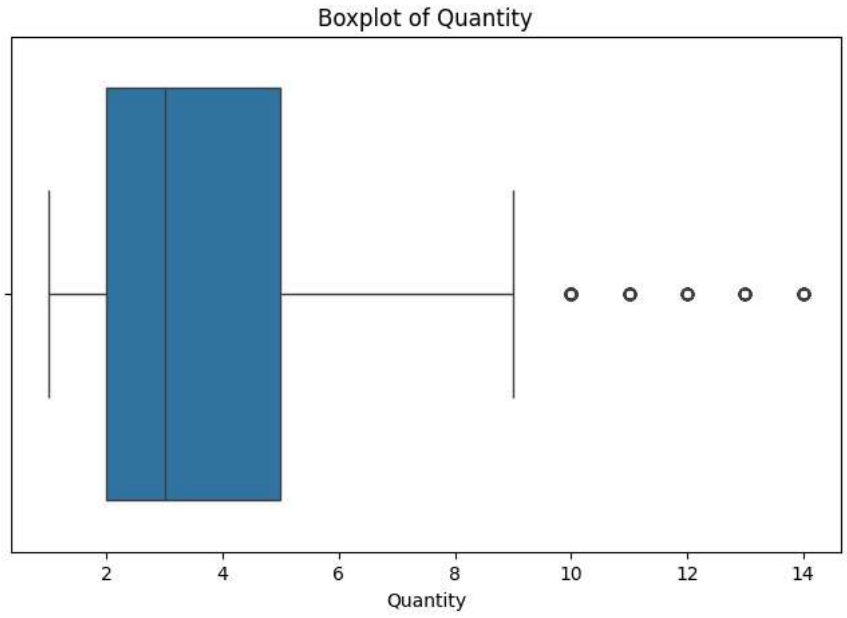



```
plt.figure(figsize=(8, 5))

sns.boxplot(x=df['Quantity'])

plt.title('Boxplot of Quantity')
plt.xlabel('Quantity')

plt.show()
```

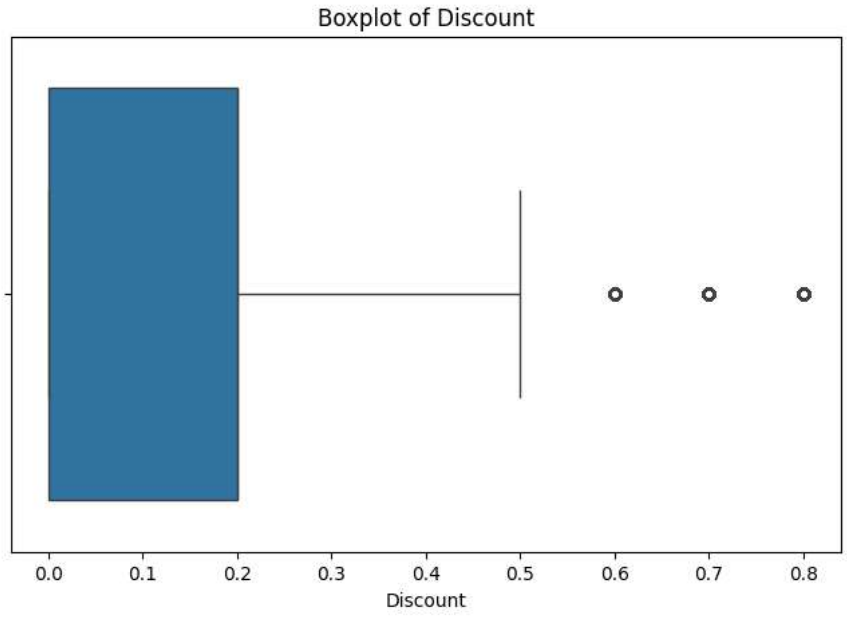



```
plt.figure(figsize=(8, 5))

sns.boxplot(x=df['Discount'])

plt.title('Boxplot of Discount')
plt.xlabel('Discount')

plt.show()
```




```
category_sales = (
    df.groupby('Category')['Sales']
    .sum()
    .sort_values(ascending=False)
)

display(category_sales)
```

Category	Sales
Technology	839893.2790
Furniture	754747.7613
Office Supplies	731893.3140

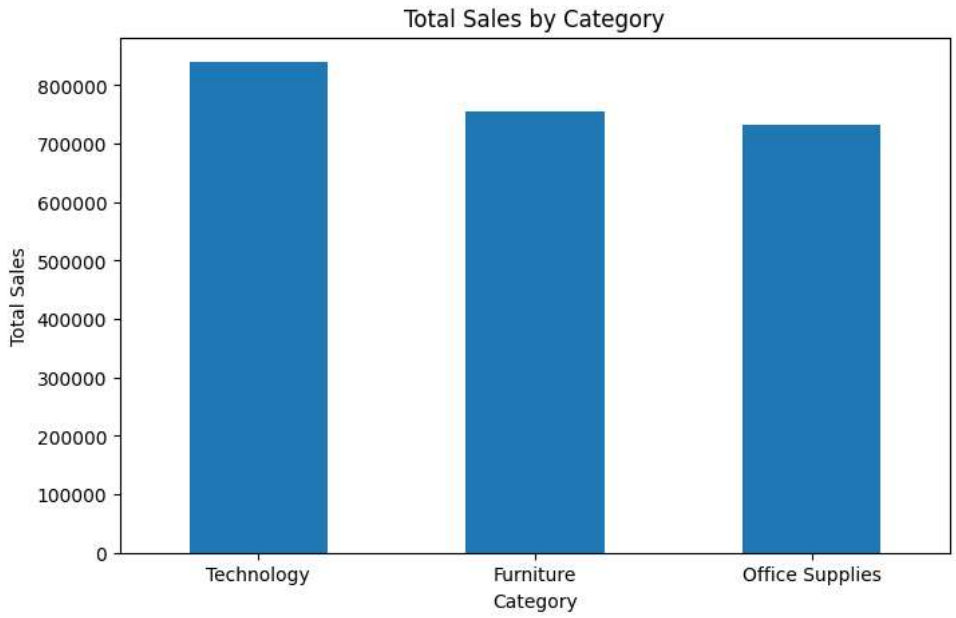
dtype: float64

```
#sales by category
plt.figure(figsize=(8, 5))

category_sales.plot(kind='bar')

plt.title('Total Sales by Category')
plt.xlabel('Category')
plt.ylabel('Total Sales')
plt.xticks(rotation=0)

plt.show()
```




```
category_profit = (
    df.groupby('Category')['Profit']
    .sum()
    .sort_values(ascending=False)
)

display(category_profit)
```

Category	Profit
Technology	146543.3756
Office Supplies	126023.4434
Furniture	19729.9956

dtype: float64

```
#category by profit
category_profit = (
    df.groupby('Category')['Profit']
    .sum()
    .sort_values(ascending=False)
)

display(category_profit)
```

Category	Profit
Technology	146543.3756
Office Supplies	126023.4434
Furniture	19729.9956

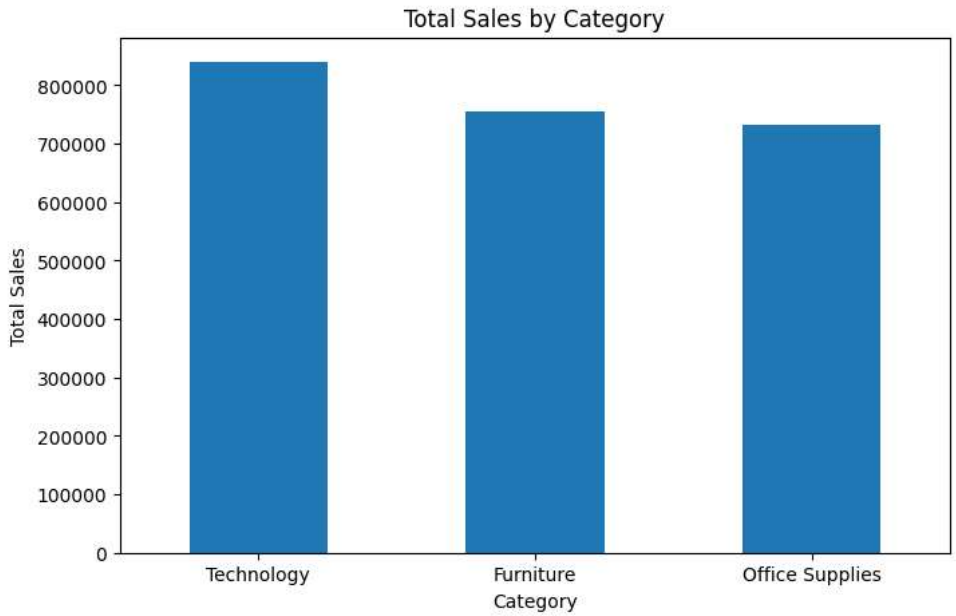
dtype: float64

```
plt.figure(figsize=(8, 5))

category_sales.plot(kind='bar')
```

```
plt.title('Total Sales by Category')
plt.xlabel('Category')
plt.ylabel('Total Sales')
plt.xticks(rotation=0)

plt.show()
```




```
category_profit = (
    df.groupby('Category')['Profit']
    .sum()
    .sort_values(ascending=False)
)

display(category_profit)
plt.figure(figsize=(8, 5))

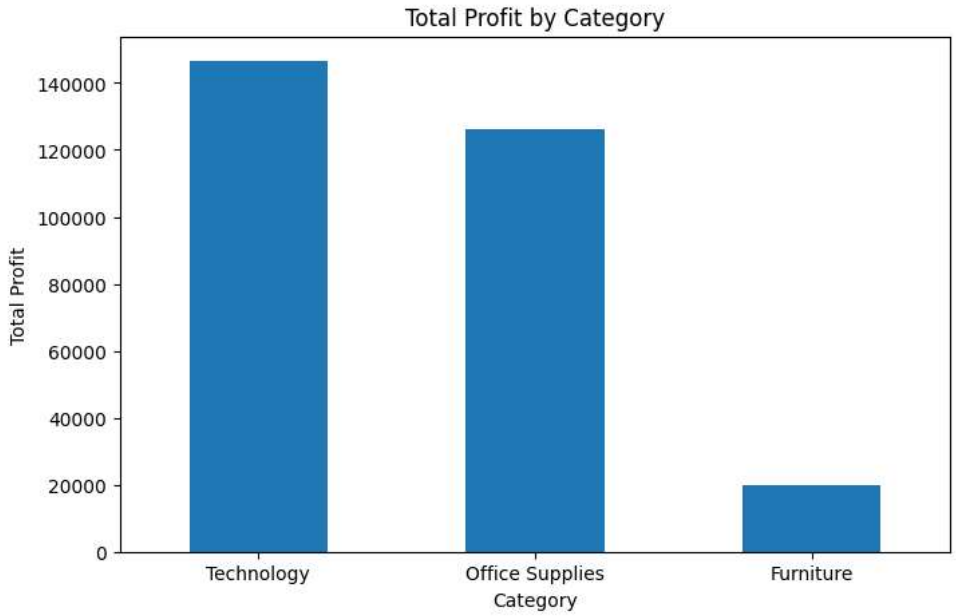
category_profit.plot(kind='bar')

plt.title('Total Profit by Category')
plt.xlabel('Category')
plt.ylabel('Total Profit')
plt.xticks(rotation=0)

plt.show()
```

Category	Profit
Technology	146543.3756
Office Supplies	126023.4434
Furniture	19729.9956

dtype: float64



Sub category Analysis

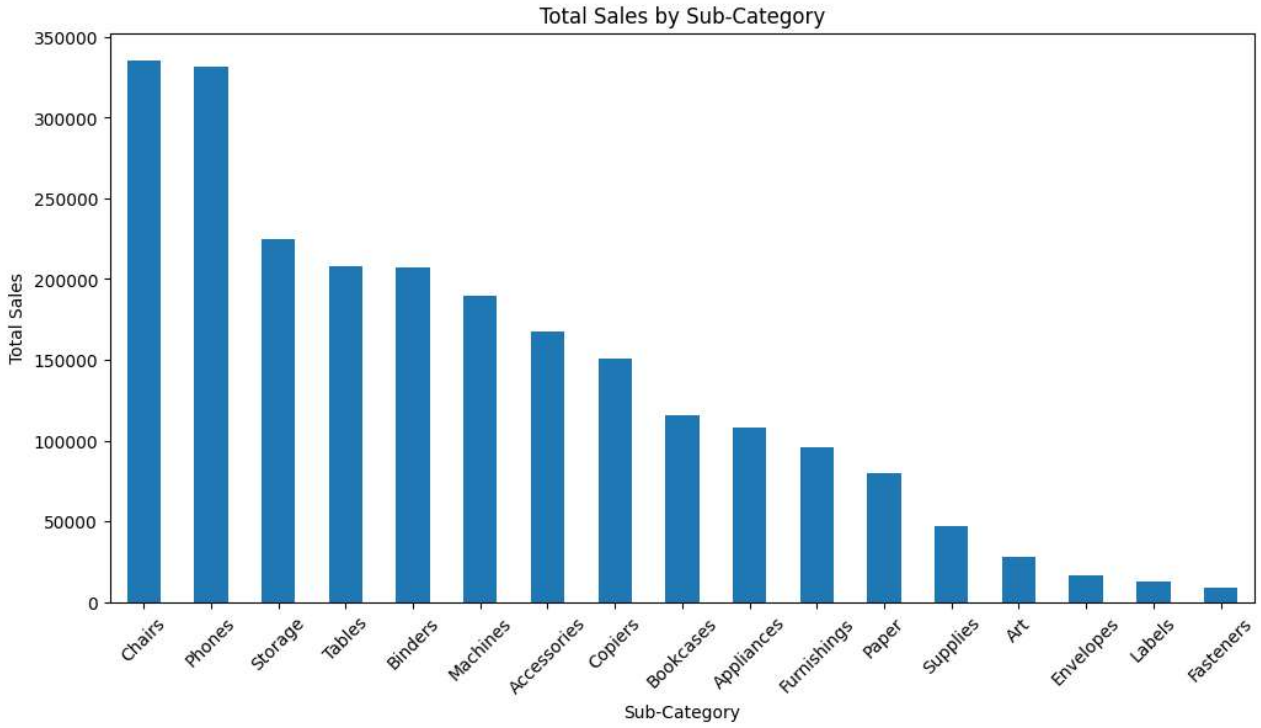
```
subcategory_sales = (
    df.groupby('Sub-Category')['Sales']
    .sum()
    .sort_values(ascending=False)
)

plt.figure(figsize=(12, 6))

subcategory_sales.plot(kind='bar')

plt.title('Total Sales by Sub-Category')
plt.xlabel('Sub-Category')
plt.ylabel('Total Sales')
plt.xticks(rotation=45)

plt.show()
```



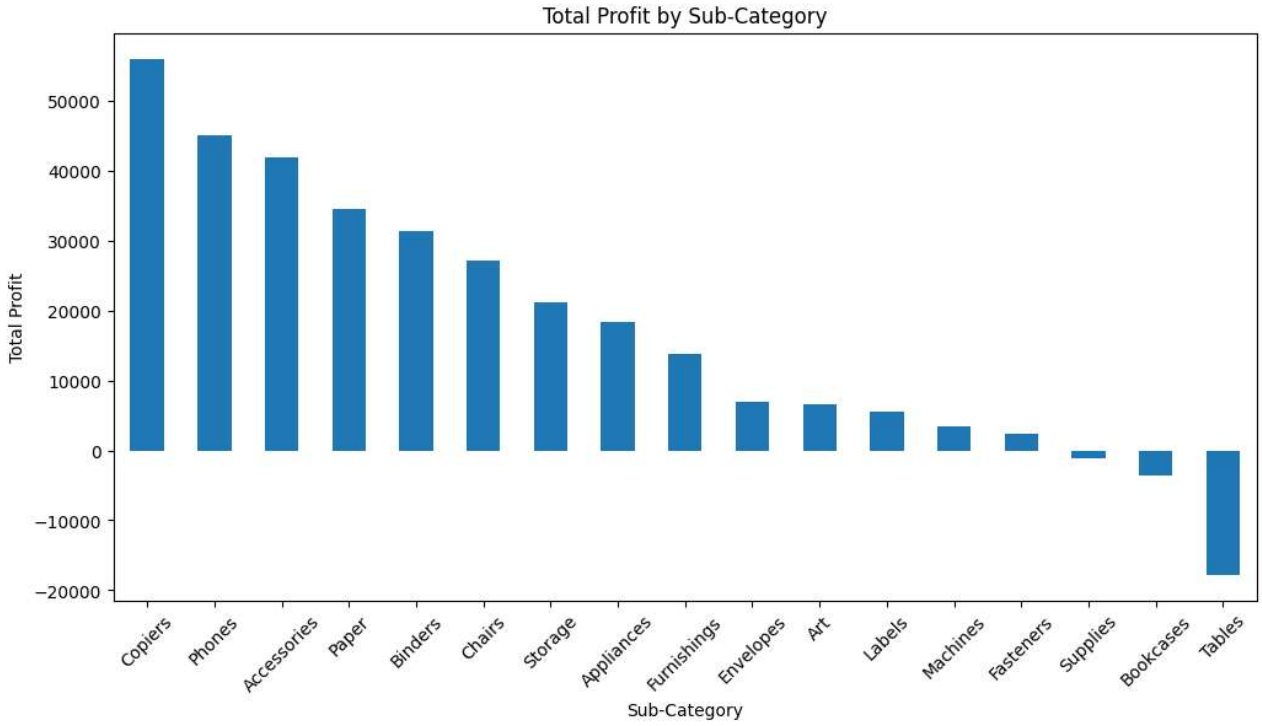

```
subcategory_profit = (
    df.groupby('Sub-Category')['Profit']
    .sum()
    .sort_values(ascending=False)
)

plt.figure(figsize=(12, 6))

subcategory_profit.plot(kind='bar')

plt.title('Total Profit by Sub-Category')
plt.xlabel('Sub-Category')
plt.ylabel('Total Profit')
plt.xticks(rotation=45)

plt.show()
```




```
region_sales = (
    df.groupby('Region')['Sales']
    .sum()
    .sort_values(ascending=False)
)

display(region_sales)
```

Region	Sales
West	739813.6085
East	691828.1680
Central	503170.6728
South	391721.9050

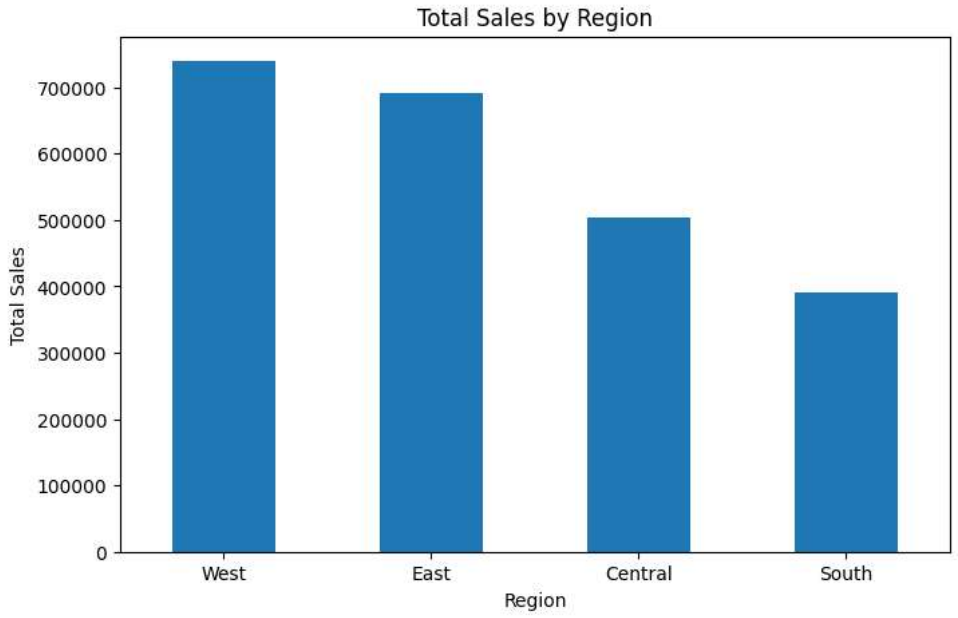
dtype: float64

```
plt.figure(figsize=(8, 5))

region_sales.plot(kind='bar')

plt.title('Total Sales by Region')
plt.xlabel('Region')
plt.ylabel('Total Sales')
plt.xticks(rotation=0)

plt.show()
```




```
region_profit = (
    df.groupby('Region')['Profit']
    .sum()
    .sort_values(ascending=False)
)

display(region_profit)
```

Region	Profit
West	110798.8170
East	94883.2603
South	46749.4303
Central	39865.3070

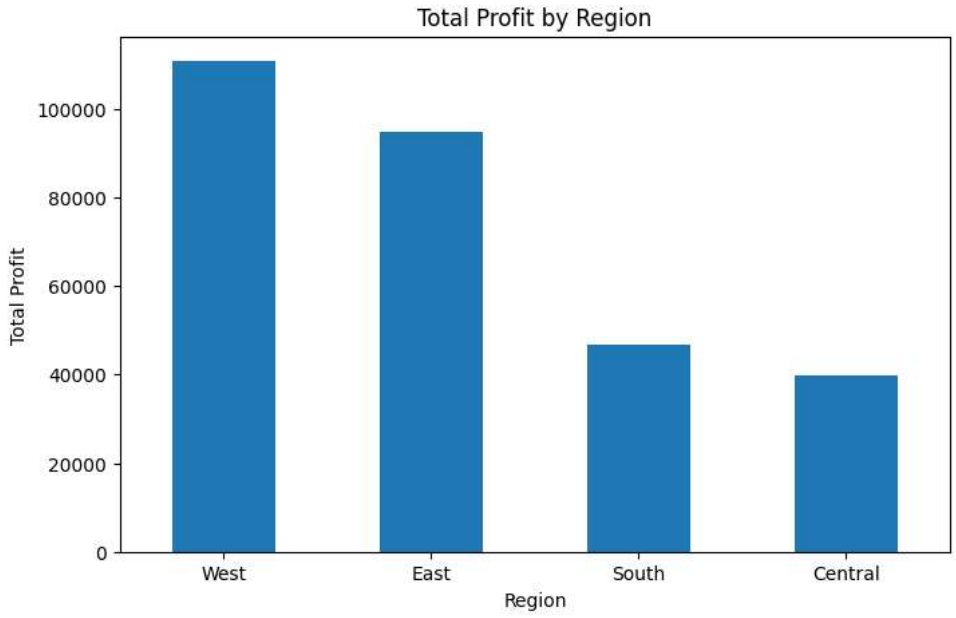
dtype: float64

```
plt.figure(figsize=(8, 5))

region_profit.plot(kind='bar')

plt.title('Total Profit by Region')
plt.xlabel('Region')
plt.ylabel('Total Profit')
plt.xticks(rotation=0)

plt.show()
```



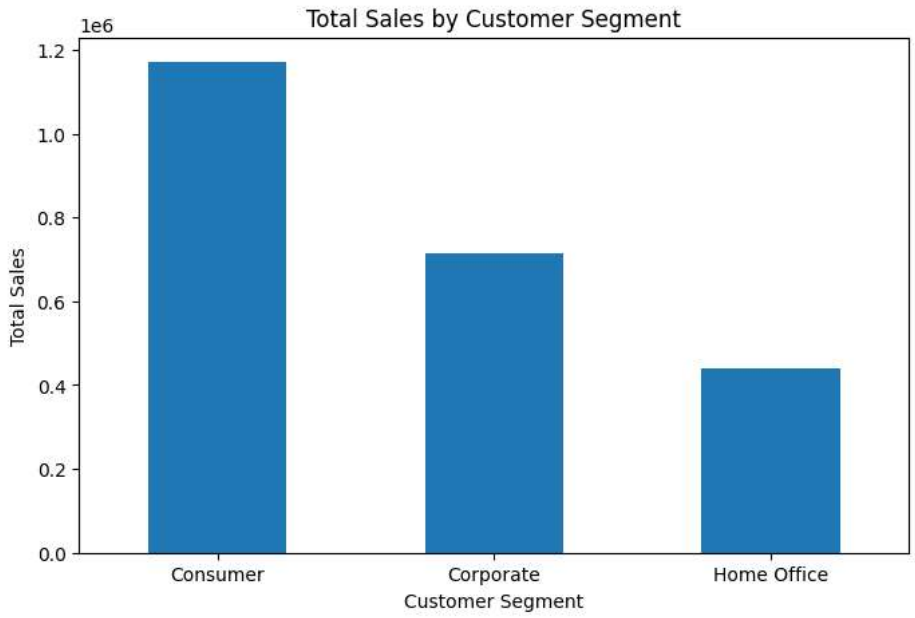

```
segment_sales = (
    df.groupby('Segment')['Sales']
    .sum()
    .sort_values(ascending=False)
)

plt.figure(figsize=(8, 5))

segment_sales.plot(kind='bar')

plt.title('Total Sales by Customer Segment')
plt.xlabel('Customer Segment')
plt.ylabel('Total Sales')
plt.xticks(rotation=0)

plt.show()
```




```
segment_profit = (
    df.groupby('Segment')['Profit']
    .sum()
    .sort_values(ascending=False)
)

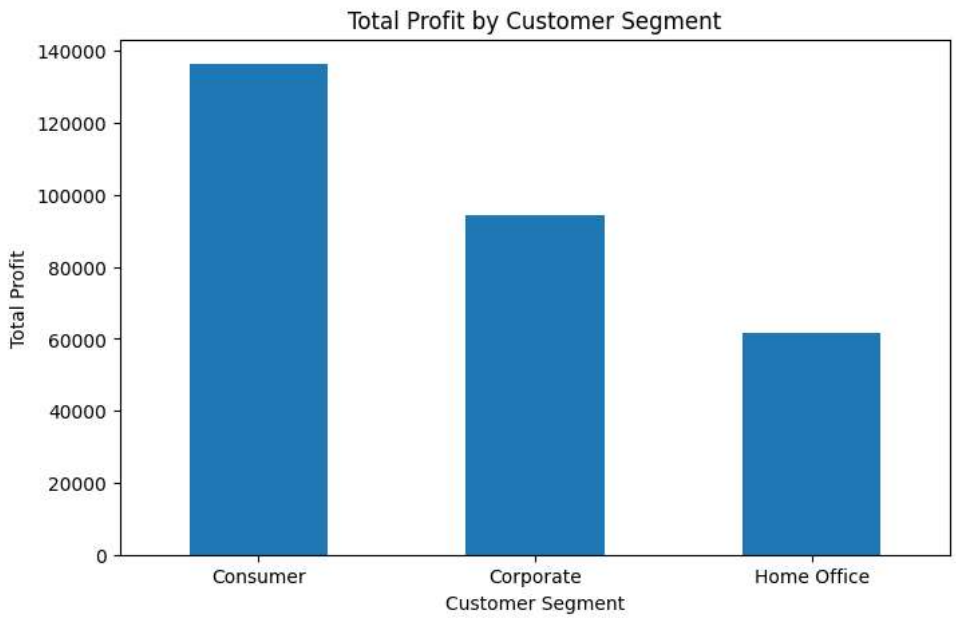
plt.figure(figsize=(8, 5))

segment_profit.plot(kind='bar')

plt.title('Total Profit by Customer Segment')
plt.xlabel('Customer Segment')
```

```
plt.ylabel('Total Profit')
plt.xticks(rotation=0)

plt.show()
```




```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x='Sales',
    y='Profit'
)

plt.title('Sales vs Profit')
plt.xlabel('Sales')
plt.ylabel('Profit')

plt.show()
```



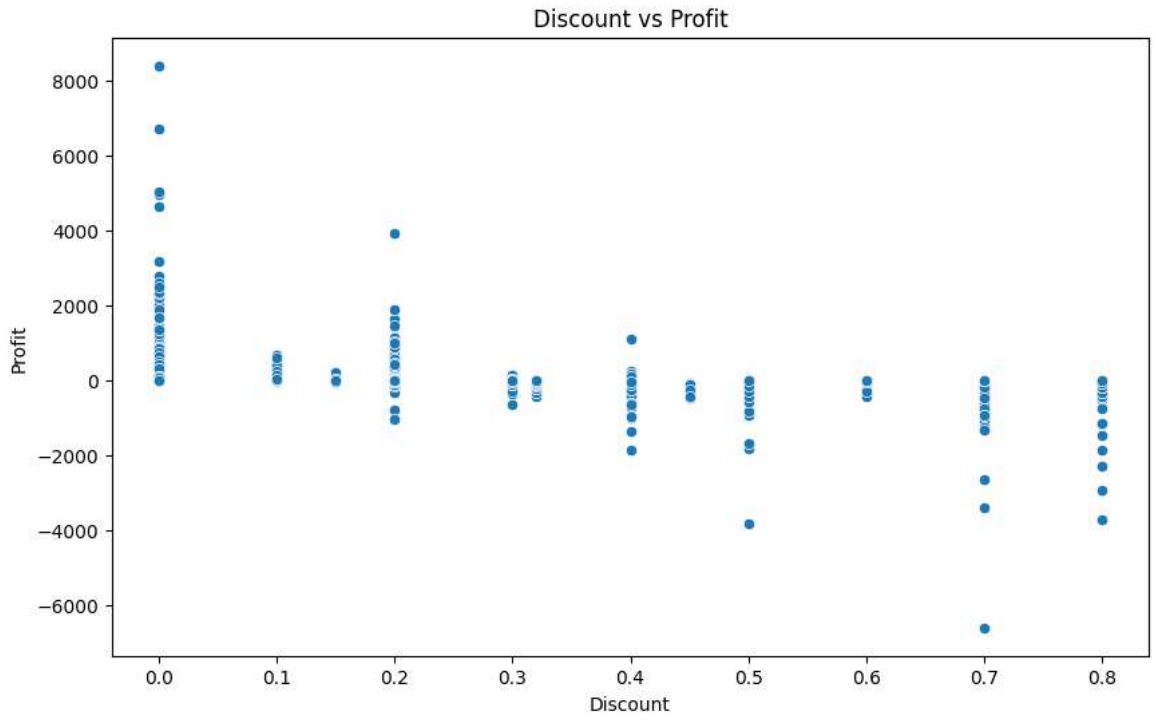

```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x='Discount',
    y='Profit'
```

```
)

plt.title('Discount vs Profit')
plt.xlabel('Discount')
plt.ylabel('Profit')

plt.show()
```

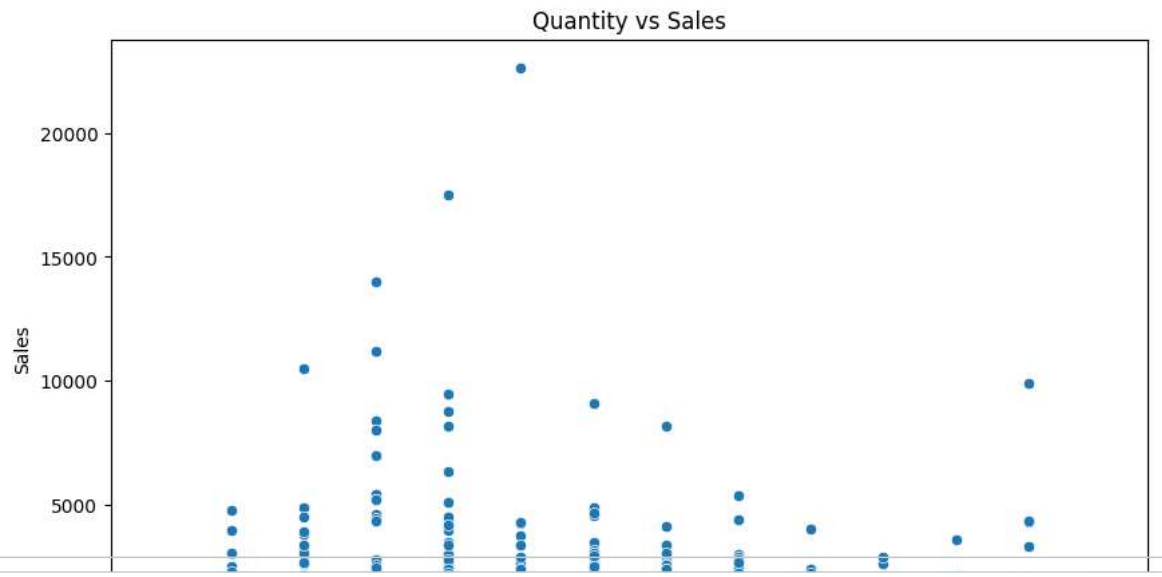



```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x='Quantity',
    y='Sales'
)

plt.title('Quantity vs Sales')
plt.xlabel('Quantity')
plt.ylabel('Sales')

plt.show()
```




```
correlation = df[
    ['Sales', 'Profit', 'Quantity', 'Discount']
].corr()

display(correlation)
```

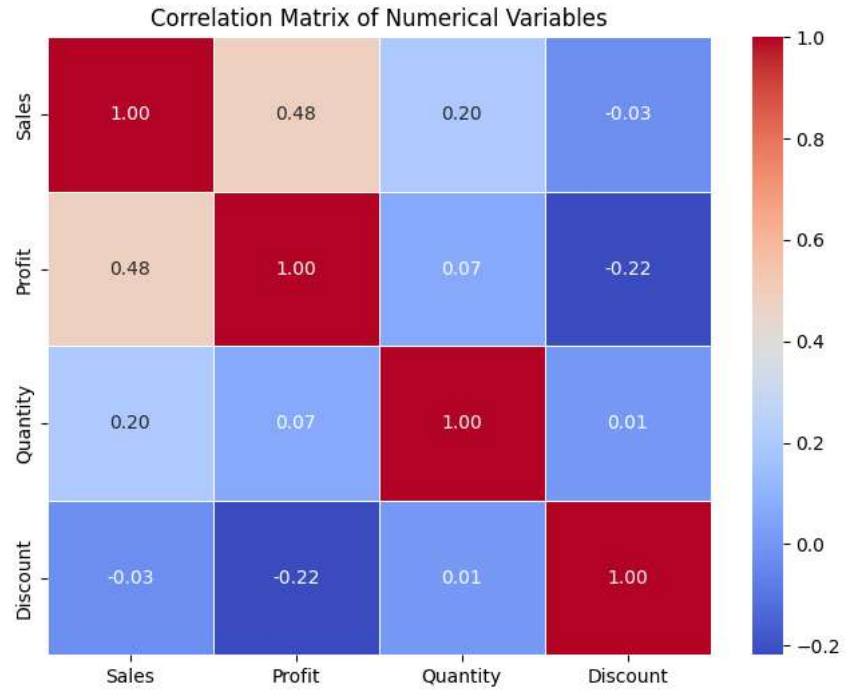
	Sales	Profit	Quantity	Discount
Sales	1.000000	0.481460	0.198457	-0.027756
Profit	0.481460	1.000000	0.066145	-0.218882
Quantity	0.198457	0.066145	1.000000	0.007475
Discount	-0.027756	-0.218882	0.007475	1.000000

```
plt.figure(figsize=(8, 6))

sns.heatmap(
    correlation,
    annot=True,
    fmt='.2f',
    cmap='coolwarm',
    linewidths=0.5
)

plt.title('Correlation Matrix of Numerical Variables')

plt.show()
```




```
monthly_sales = (
    df.groupby('Order Month')['Sales']
    .sum()
)

display(monthly_sales.head(10))
```

Order Month	Sales
2023-01	14518.0550
2023-02	4519.8920
2023-03	56933.9090
2023-04	28295.3450
2023-05	26319.7670
2023-06	34669.4796
2023-07	33946.3930
2023-08	28918.3385
2023-09	82670.4288
2023-10	32413.3390

dtype: float64

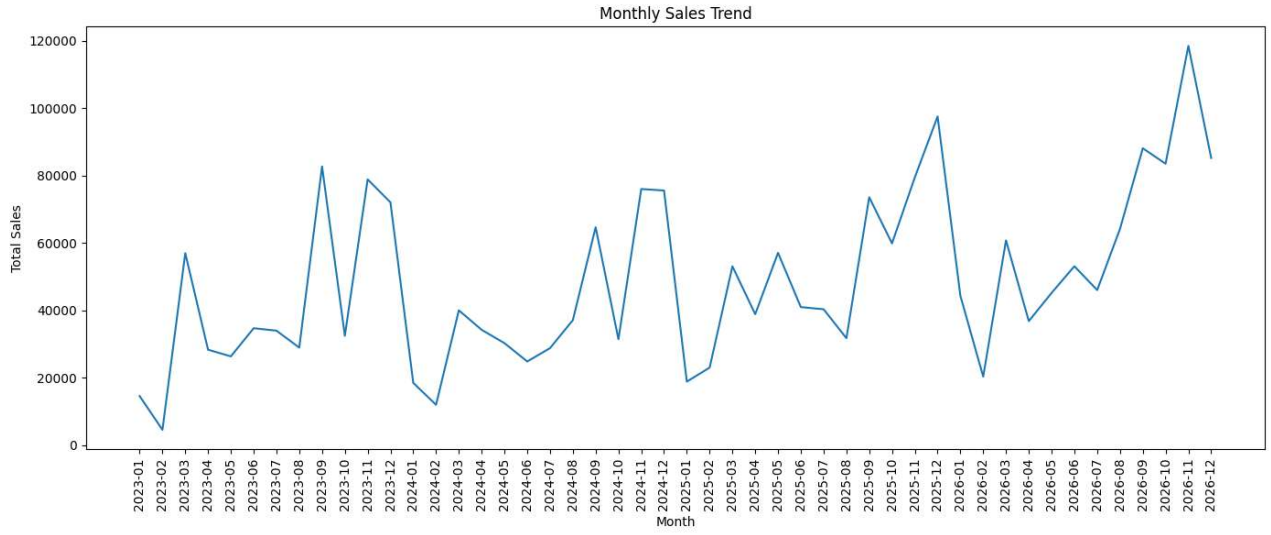
```
plt.figure(figsize=(14, 6))

plt.plot(
    monthly_sales.index.astype(str),
    monthly_sales.values
)

plt.title('Monthly Sales Trend')
plt.xlabel('Month')
plt.ylabel('Total Sales')

plt.xticks(rotation=90)

plt.tight_layout()
plt.show()
```




```
yearly_sales = (
    df.groupby('Year')['Sales']
    .sum()
)

display(yearly_sales)
```

Year	Sales
2023	494040.2121
2024	472993.0310
2025	613933.5800
2026	745567.5312

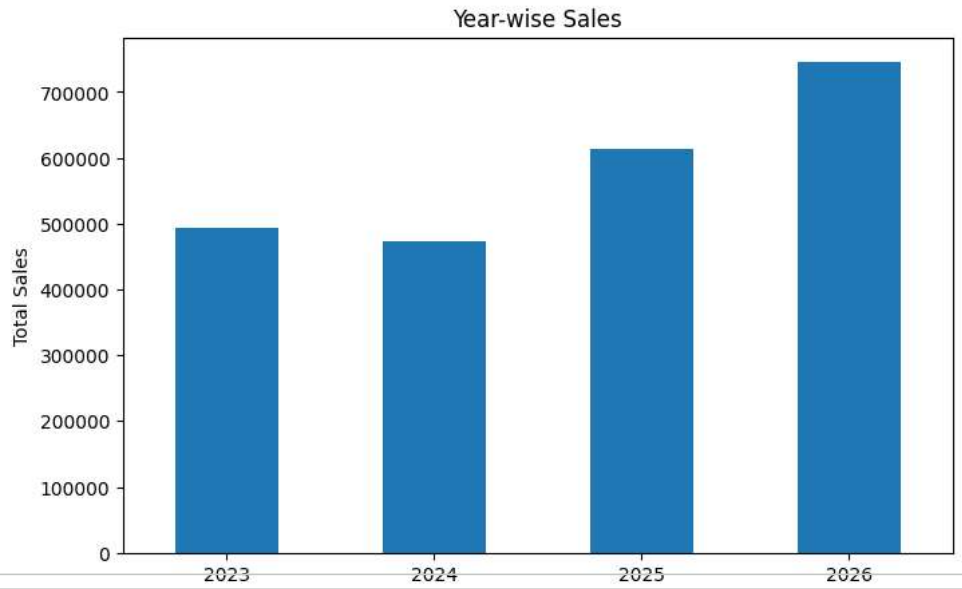
dtype: float64

```
plt.figure(figsize=(8, 5))

yearly_sales.plot(kind='bar')

plt.title('Year-wise Sales')
plt.xlabel('Year')
plt.ylabel('Total Sales')
plt.xticks(rotation=0)

plt.show()
```




```
quarterly_sales = (
    df.groupby('Quarter')['Sales']
    .sum()
)

display(quarterly_sales)
```

Quarter	Sales
1	366492.8538
2	450323.7796
3	619707.2977
4	890010.4232

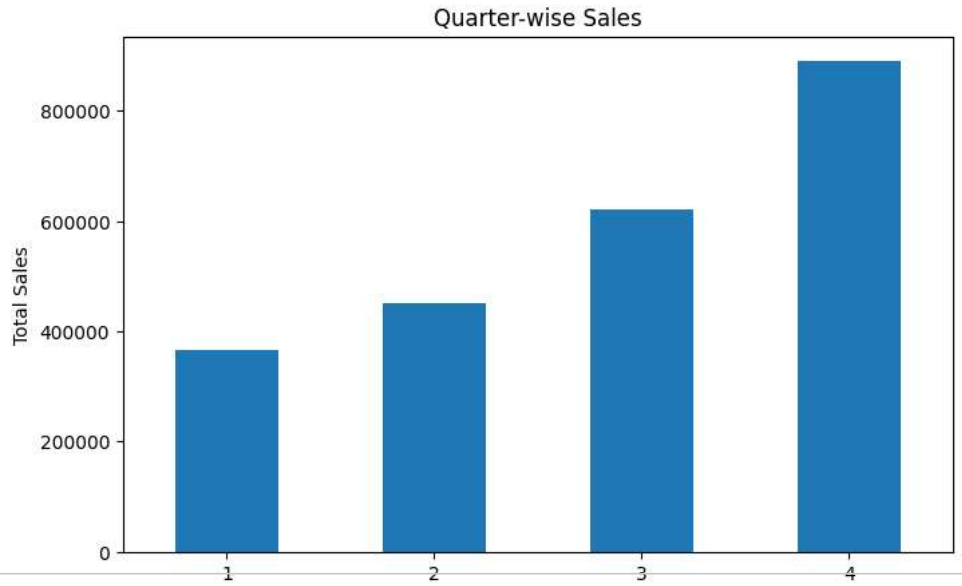
dtype: float64

```
plt.figure(figsize=(8, 5))

quarterly_sales.plot(kind='bar')

plt.title('Quarter-wise Sales')
plt.xlabel('Quarter')
plt.ylabel('Total Sales')
plt.xticks(rotation=0)

plt.show()
```




```
print("Profit Margin Statistics:")

display(
    df['Profit Margin'].describe()
)
```

```
Profit Margin Statistics:
```

	Profit Margin
count	10194.000000
mean	12.221784
std	46.488992
min	-275.000000
25%	7.500000
50%	27.000000
75%	36.250000
max	50.000000

dtype: float64

```
category_margin = (
    df.groupby('Category')['Profit Margin']
    .mean()
    .sort_values(ascending=False)
)

display(category_margin)
```

Category	Profit Margin
Technology	15.768028
Office Supplies	13.999902
Furniture	4.266289

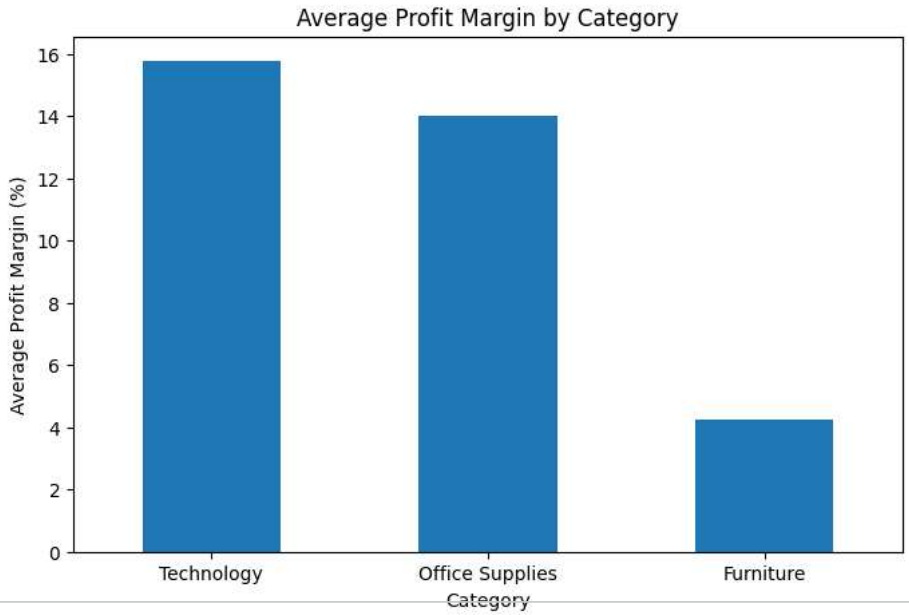
dtype: float64

```
plt.figure(figsize=(8, 5))

category_margin.plot(kind='bar')

plt.title('Average Profit Margin by Category')
plt.xlabel('Category')
plt.ylabel('Average Profit Margin (%)')
plt.xticks(rotation=0)

plt.show()
```




```
eda_summary = pd.DataFrame({
    'Metric': [
        'Total Sales',
        'Total Profit',
        'Average Sales',
        'Average Profit',
        'Average Quantity',
        'Average Discount',
        'Average Profit Margin',
        'Highest Sales Category',
        'Highest Profit Category',
        'Highest Sales Region',
        'Highest Profit Region'
    ],
    'Value': [
        df['Sales'].sum(),
        df['Profit'].sum(),
        df['Sales'].mean(),
        df['Profit'].mean(),
        df['Quantity'].mean(),
        df['Discount'].mean(),
        df['Profit Margin'].mean(),
        category_sales.idxmax(),
        category_profit.idxmax(),
        region_sales.idxmax(),
        region_profit.idxmax()
    ]
})

display(eda_summary)
```

	Metric	Value
0	Total Sales	2326534.3543
1	Total Profit	292296.8146
2	Average Sales	228.225854
3	Average Profit	28.673417
4	Average Quantity	3.791838
5	Average Discount	0.155385
6	Average Profit Margin	12.221784
7	Highest Sales Category	Technology
8	Highest Profit Category	Technology
9	Highest Sales Region	West
10	Highest Profit Region	West

```
# ============================================================
# PART 4 — DESCRIPTIVE STATISTICS
# ============================================================
```

```
numerical_cols = ['Sales', 'Profit', 'Quantity', 'Discount', 'Profit Margin']

descriptive_stats = pd.DataFrame(index=numerical_cols)

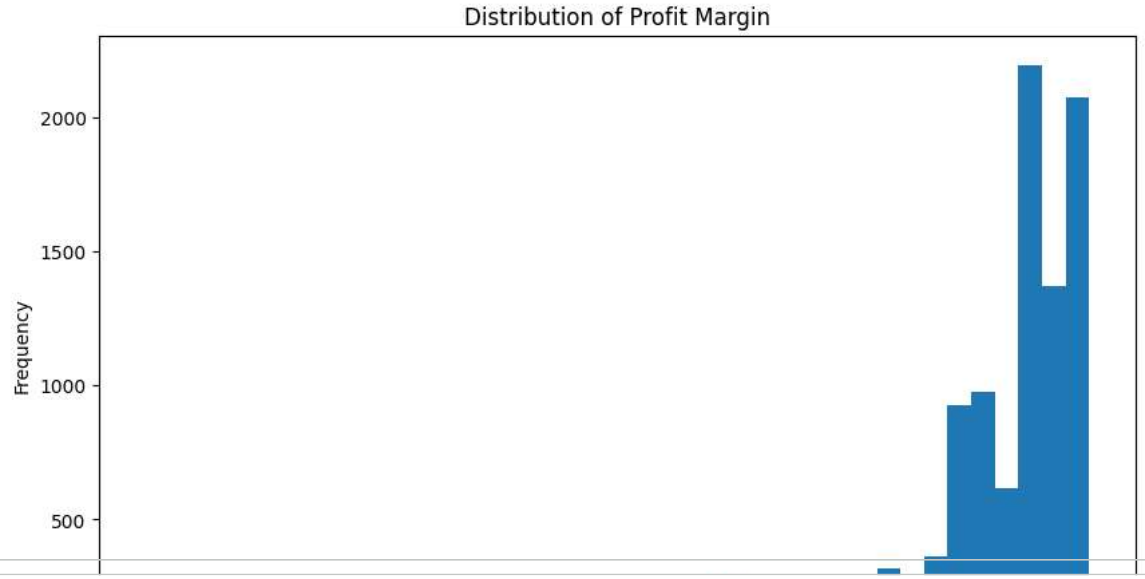
descriptive_stats['Mean'] = df[numerical_cols].mean()
descriptive_stats['Median'] = df[numerical_cols].median()
descriptive_stats['Mode'] = df[numerical_cols].mode().iloc[0]
descriptive_stats['Variance'] = df[numerical_cols].var()
descriptive_stats['Std Deviation'] = df[numerical_cols].std()
descriptive_stats['Minimum'] = df[numerical_cols].min()
descriptive_stats['Maximum'] = df[numerical_cols].max()
descriptive_stats['Range'] = (
    df[numerical_cols].max() - df[numerical_cols].min()
)
descriptive_stats['IQR'] = (
    df[numerical_cols].quantile(0.75)
    - df[numerical_cols].quantile(0.25)
)
descriptive_stats['Skewness'] = df[numerical_cols].skew()
descriptive_stats['Kurtosis'] = df[numerical_cols].kurtosis()

display(descriptive_stats.round(4))
```

	Mean	Median	Mode	Variance	Std Deviation	Minimum	Maximum	Range	IQR	Skewness	Kurtosis
Sales	228.2259	53.91	12.96	384284.4888	619.9068	0.444	22638.480	22638.036	192.2800	12.9839	306.3663
Profit	28.6734	8.69	0.00	54040.0297	232.4651	-6599.978	8399.976	14999.954	27.5371	7.6049	401.6929
Quantity	3.7918	3.00	2.00	4.9654	2.2283	1.000	14.000	13.000	3.0000	1.2686	1.9615
Discount	0.1554	0.20	0.00	0.0425	0.2062	0.000	0.800	0.800	0.2000	1.6875	2.4163

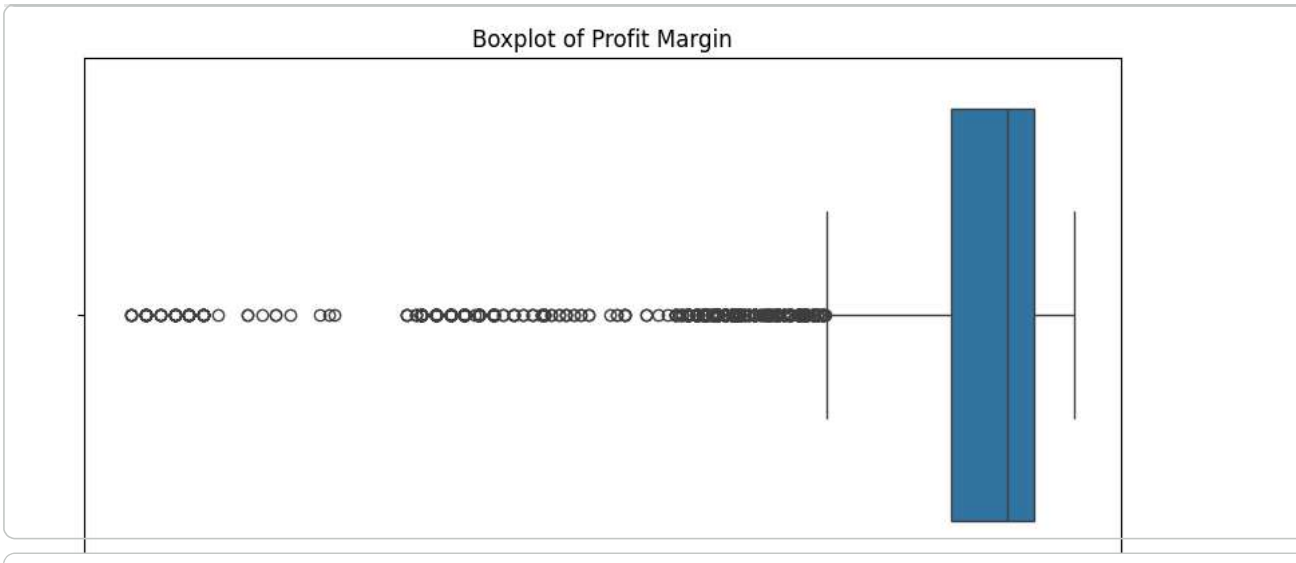
```
# Profit Margin Histogram

plt.figure(figsize=(10, 6))
plt.hist(df['Profit Margin'], bins=40)
plt.title('Distribution of Profit Margin')
plt.xlabel('Profit Margin (%)')
plt.ylabel('Frequency')
plt.show()
```




```
# Profit Margin Boxplot

plt.figure(figsize=(10, 5))
sns.boxplot(x=df['Profit Margin'])
plt.title('Boxplot of Profit Margin')
plt.xlabel('Profit Margin (%)')
plt.show()
```




```
# ============================================================
# PART 5.1 — 95% CONFIDENCE INTERVAL FOR MEAN SALES
# ============================================================

from scipy import stats
import numpy as np

sales = df['Sales'].dropna()

n = len(sales)
mean_sales = sales.mean()
std_sales = sales.std()

confidence_level = 0.95
alpha = 1 - confidence_level

# t-critical value
t_critical = stats.t.ppf(1 - alpha/2, df=n-1)

# Standard error
standard_error = std_sales / np.sqrt(n)

# Margin of error
margin_error = t_critical * standard_error

# Confidence interval
lower_ci = mean_sales - margin_error
upper_ci = mean_sales + margin_error

print("95% Confidence Interval for Mean Sales")
print("-" * 50)
print(f"Sample Size       : {n}")
print(f"Mean Sales        : {mean_sales:.4f}")
print(f"Standard Error    : {standard_error:.4f}")
print(f"Margin of Error   : {margin_error:.4f}")
print(f"Lower Limit       : {lower_ci:.4f}")
print(f"Upper Limit       : {upper_ci:.4f}")
print("-" * 50)
```

```
95% Confidence Interval for Mean Sales
--------------------------------------------------
Sample Size       : 10194
Mean Sales        : 228.2259
Standard Error    : 6.1398
Margin of Error   : 12.0352
Lower Limit       : 216.1906
Upper Limit       : 240.2611
--------------------------------------------------
```

```
# ============================================================
# PART 5.2 — INDEPENDENT T-TEST
# Consumer vs Corporate
# ============================================================

consumer_sales = df.loc[
    df['Segment'] == 'Consumer',
    'Sales'
].dropna()

corporate_sales = df.loc[
```

```
    df['Segment'] == 'Corporate',
    'Sales'
].dropna()

print("Consumer observations :", len(consumer_sales))
print("Corporate observations:", len(corporate_sales))

print("\nMean Sales")
print("Consumer :", consumer_sales.mean())
print("Corporate:", corporate_sales.mean())
```

```
Consumer observations : 5281
Corporate observations: 3090

Mean Sales
Consumer : 221.6738850596478
Corporate: 231.6524708090615
```

```
# Welch's Independent t-test

t_stat, p_value = stats.ttest_ind(
    consumer_sales,
    corporate_sales,
    equal_var=False
)

alpha = 0.05

print("Independent t-test Results")
print("-" * 50)
print(f"t-statistic : {t_stat:.4f}")
print(f"p-value     : {p_value:.6f}")
print(f"Alpha       : {alpha}")

if p_value < alpha:
    print("\nDecision: Reject H0")
    print("Conclusion: There is a statistically significant difference "
          "in mean Sales between Consumer and Corporate customers.")
else:
    print("\nDecision: Fail to Reject H0")
    print("Conclusion: There is not enough statistical evidence "
          "of a difference in mean Sales between Consumer and Corporate customers.")
```

```
Independent t-test Results
--------------------------------------------------
t-statistic : -0.7428
p-value     : 0.457606
Alpha       : 0.05

Decision: Fail to Reject H0
Conclusion: There is not enough statistical evidence of a difference in mean Sales between Consumer and Corporate customers.
```

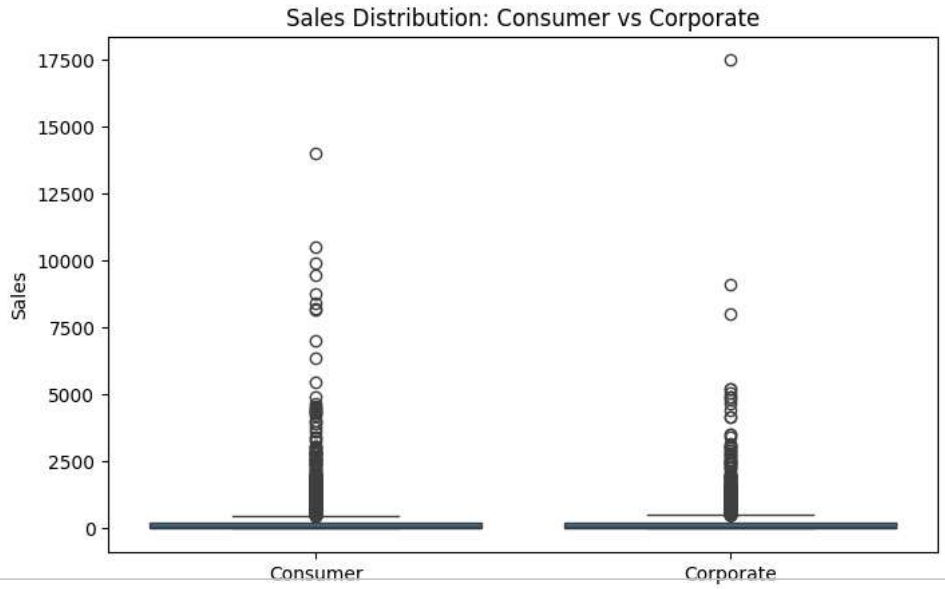
```
# Visual comparison of Consumer and Corporate Sales

ttest_data = df[
    df['Segment'].isin(['Consumer', 'Corporate'])
]

plt.figure(figsize=(8, 5))

sns.boxplot(
    data=ttest_data,
    x='Segment',
    y='Sales'
)

plt.title('Sales Distribution: Consumer vs Corporate')
plt.xlabel('Customer Segment')
plt.ylabel('Sales')
plt.show()
```




```
# ============================================================
# PART 5.3 — ONE-WAY ANOVA
# Profit across Product Categories
# ============================================================

furniture_profit = df.loc[
    df['Category'] == 'Furniture',
    'Profit'
].dropna()

office_profit = df.loc[
    df['Category'] == 'Office Supplies',
    'Profit'
].dropna()

technology_profit = df.loc[
    df['Category'] == 'Technology',
    'Profit'
].dropna()

print("Number of observations:")
print("Furniture       :", len(furniture_profit))
print("Office Supplies :", len(office_profit))
print("Technology      :", len(technology_profit))

print("\nMean Profit:")
print("Furniture       :", furniture_profit.mean())
print("Office Supplies :", office_profit.mean())
print("Technology      :", technology_profit.mean())
```

```
Number of observations:
Furniture       : 2201
Office Supplies : 6128
Technology      : 1865

Mean Profit:
Furniture       : 8.964105224897764
Office Supplies : 20.565183322454303
Technology      : 78.5755365147453
```

```
# One-way ANOVA

f_stat, p_value_anova = stats.f_oneway(
    furniture_profit,
    office_profit,
    technology_profit
)

alpha = 0.05

print("One-Way ANOVA Results")
print("-" * 50)
print(f"F-statistic : {f_stat:.4f}")
print(f"p-value     : {p_value_anova:.6f}")
print(f"Alpha       : {alpha}")

if p_value_anova < alpha:
    print("\nDecision: Reject H0")
```

```
    print("Conclusion: There is a statistically significant "
          "difference in mean Profit among the product categories.")
else:
    print("\nDecision: Fail to Reject H0")
    print("Conclusion: There is not enough statistical evidence "
          "that mean Profit differs among the product categories.")
```

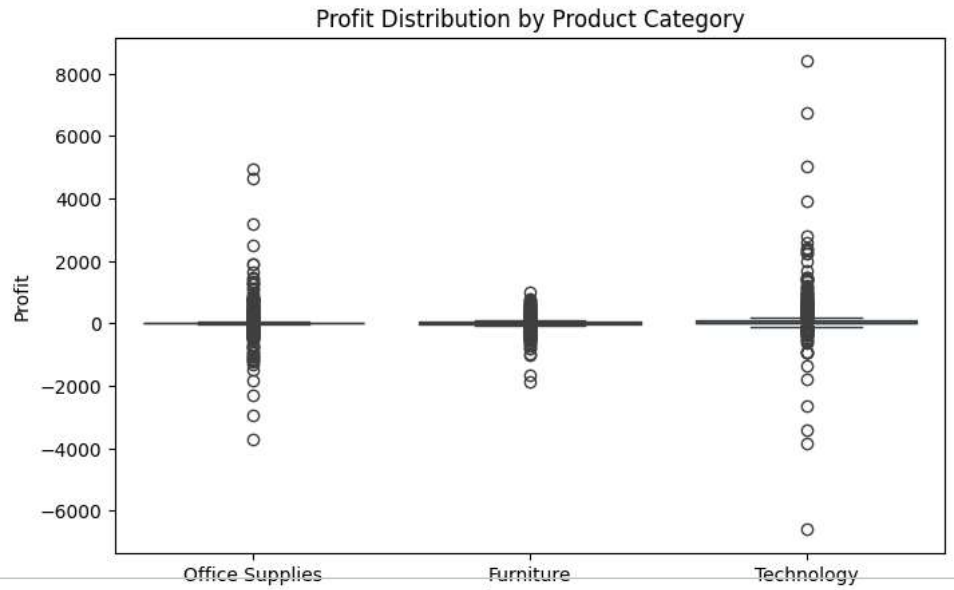
```
One-Way ANOVA Results
--------------------------------------------------
F-statistic : 55.1895
p-value     : 0.000000
Alpha       : 0.05

Decision: Reject H0
Conclusion: There is a statistically significant difference in mean Profit among the product categories.
```

```
plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x='Category',
    y='Profit'
)

plt.title('Profit Distribution by Product Category')
plt.xlabel('Product Category')
plt.ylabel('Profit')
plt.show()
```




```
# ============================================================
# PART 5.4 — CHI-SQUARE TEST OF INDEPENDENCE
# Segment vs Category
# ============================================================

contingency_table = pd.crosstab(
    df['Segment'],
    df['Category']
)

print("Contingency Table")
display(contingency_table)
```

Contingency Table

Segment	Furniture	Office Supplies	Technology
Consumer	1143	3187	951
Corporate	678	1848	564
Home Office	380	1093	350

```
# ============================================================
# PART 5.4 — CHI-SQUARE TEST OF INDEPENDENCE
# Segment vs Category
# ============================================================
```

```
contingency_table = pd.crosstab(
    df['Segment'],
    df['Category']
)

print("Contingency Table")
display(contingency_table)
```

```
Contingency Table
```

Segment	Furniture	Office Supplies	Technology
Consumer	1143	3187	951
Corporate	678	1848	564
Home Office	380	1093	350

```
chi2_stat, chi2_p, dof, expected = stats.chi2_contingency(
    contingency_table
)

alpha = 0.05

print("Chi-Square Test Results")
print("-" * 50)
print(f"Chi-square statistic : {chi2_stat:.4f}")
print(f"Degrees of freedom  : {dof}")
print(f"p-value             : {chi2_p:.6f}")
print(f"Alpha               : {alpha}")

if chi2_p < alpha:
    print("\nDecision: Reject H0")
    print("Conclusion: Customer Segment and Product Category "
          "are statistically associated.")
else:
    print("\nDecision: Fail to Reject H0")
    print("Conclusion: There is not enough statistical evidence "
          "of an association between Customer Segment and Product Category.")
```

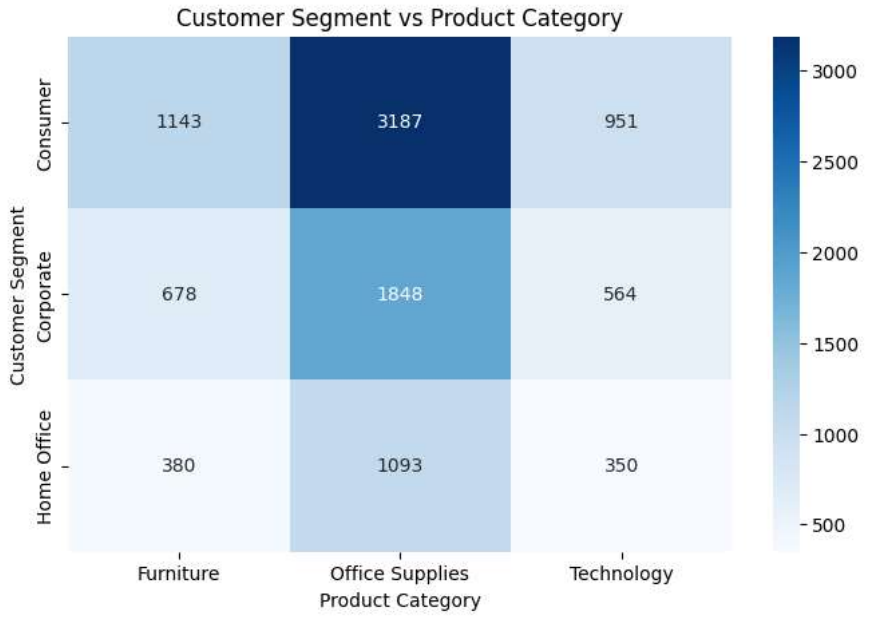
```
Chi-Square Test Results
--------------------------------------------------
Chi-square statistic : 1.8131
Degrees of freedom  : 4
p-value             : 0.770084
Alpha               : 0.05

Decision: Fail to Reject H0
Conclusion: There is not enough statistical evidence of an association between Customer Segment and Product Category.
```

```
plt.figure(figsize=(8, 5))

sns.heatmap(
    contingency_table,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title('Customer Segment vs Product Category')
plt.xlabel('Product Category')
plt.ylabel('Customer Segment')
plt.show()
```




```
# ============================================================
# FINAL INFERENTIAL STATISTICS SUMMARY
# ============================================================

inferential_summary = pd.DataFrame({
    'Test': [
        '95% Confidence Interval',
        'Independent t-test',
        'One-way ANOVA',
        'Chi-square Test'
    ],
    'Question': [
        'What is the plausible range for mean Sales?',
        'Do Consumer and Corporate customers differ in mean Sales?',
        'Does mean Profit differ across Categories?',
        'Is Segment associated with Category?'
    ],
    'Statistic': [
        f't-critical = {t_critical:.4f}',
        f't = {t_stat:.4f}',
        f'F = {f_stat:.4f}',
        f'Chi-square = {chi2_stat:.4f}'
    ],
    'p-value': [
        '-',
        p_value,
        p_value_anova,
        chi2_p
    ],
    'Significance': [
        '-',
        'Significant' if p_value < 0.05 else 'Not Significant',
        'Significant' if p_value_anova < 0.05 else 'Not Significant',
        'Significant' if chi2_p < 0.05 else 'Not Significant'
    ]
})

display(inferential_summary)
```

	Test	Question	Statistic	p-value	Significance
0	95% Confidence Interval	What is the plausible range for mean Sales?	t-critical = 1.9602	-	-
1	Independent t-test	Do Consumer and Corporate customers differ in ...	t = -0.7428	0.457606	Not Significant
2	One-way ANOVA	Does mean Profit differ across Categories?	F = 55.1895	0.0	Significant
3	Chi-square Test	Is Segment associated with Category?	Chi-square = 1.8131	0.770084	Not Significant

```
# ============================================================
# TUKEY HSD POST-HOC TEST
```

```
# ============================================================

from statsmodels.stats.multicomp import pairwise_tukeyhsd

tukey_result = pairwise_tukeyhsd(
    endog=df['Profit'],
    groups=df['Category'],
    alpha=0.05
)

print(tukey_result)
```

```
         Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====================================================================
     group1          group2     meandiff p-adj   lower   upper  reject
---------------------------------------------------------------------
      Furniture Office Supplies  11.6011 0.1077 -1.8685 25.0707  False
      Furniture      Technology  69.6114    0.0 52.5521 86.6707   True
Office Supplies      Technology  58.0104    0.0 43.6758 72.3449   True
---------------------------------------------------------------------
```

```
# ============================================================
# PART 6.1 — SIMPLE LINEAR REGRESSION
# Quantity -> Sales
# ============================================================

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

X_simple = df[['Quantity']]
y = df['Sales']

X_train_simple, X_test_simple, y_train_simple, y_test_simple = train_test_split(
    X_simple,
    y,
    test_size=0.20,
    random_state=42
)

print("Training samples:", len(X_train_simple))
print("Testing samples :", len(X_test_simple))
```

```
Training samples: 8155
Testing samples : 2039
```

```
# Train Simple Linear Regression

simple_model = LinearRegression()

simple_model.fit(
    X_train_simple,
    y_train_simple
)

simple_pred = simple_model.predict(
    X_test_simple
)

print("Intercept:", simple_model.intercept_)
print("Quantity coefficient:", simple_model.coef_[0])
```

```
Intercept: 19.49353907302705
Quantity coefficient: 54.02136431123193
```

```
# ============================================================
# SIMPLE LINEAR REGRESSION EVALUATION
# ============================================================

simple_mae = mean_absolute_error(
    y_test_simple,
    simple_pred
)

simple_mse = mean_squared_error(
    y_test_simple,
    simple_pred
)
```

```
simple_rmse = np.sqrt(simple_mse)

simple_r2 = r2_score(
    y_test_simple,
    simple_pred
)

print("Simple Linear Regression Results")
print("-" * 50)
print(f"MAE  : {simple_mae:.4f}")
print(f"MSE  : {simple_mse:.4f}")
print(f"RMSE : {simple_rmse:.4f}")
print(f"R²   : {simple_r2:.4f}")
```

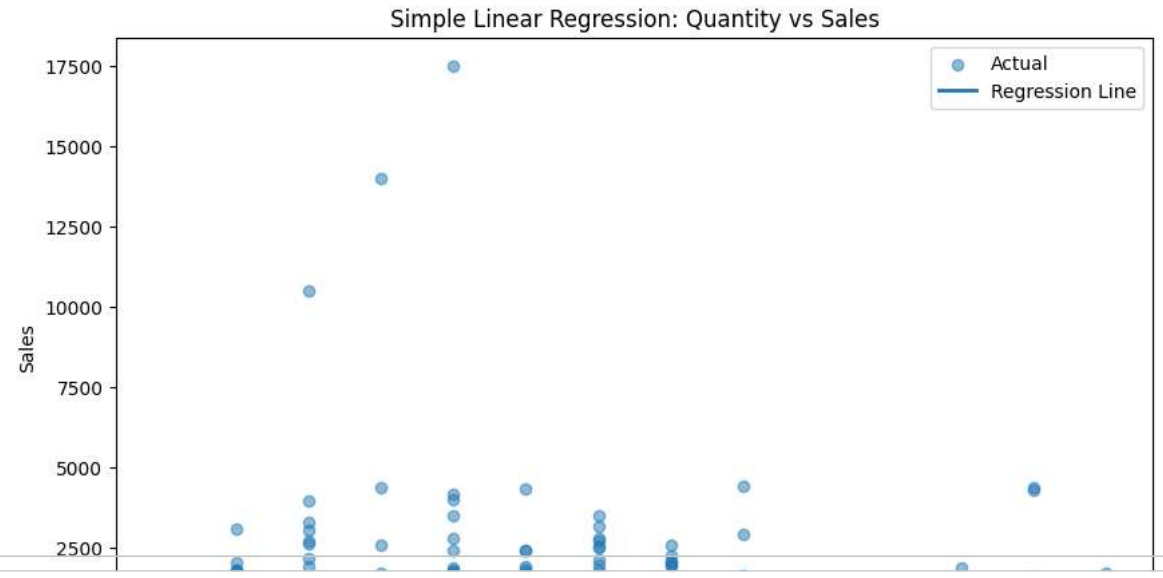
```
Simple Linear Regression Results
--------------------------------------------------
MAE  : 266.3376
MSE  : 504020.3594
RMSE : 709.9439
R²   : 0.0345
```

```
plt.figure(figsize=(10, 6))

plt.scatter(
    X_test_simple['Quantity'],
    y_test_simple,
    alpha=0.5,
    label='Actual'
)

plt.plot(
    X_test_simple['Quantity'],
    simple_pred,
    linewidth=2,
    label='Regression Line'
)

plt.title('Simple Linear Regression: Quantity vs Sales')
plt.xlabel('Quantity')
plt.ylabel('Sales')
plt.legend()
plt.show()
```

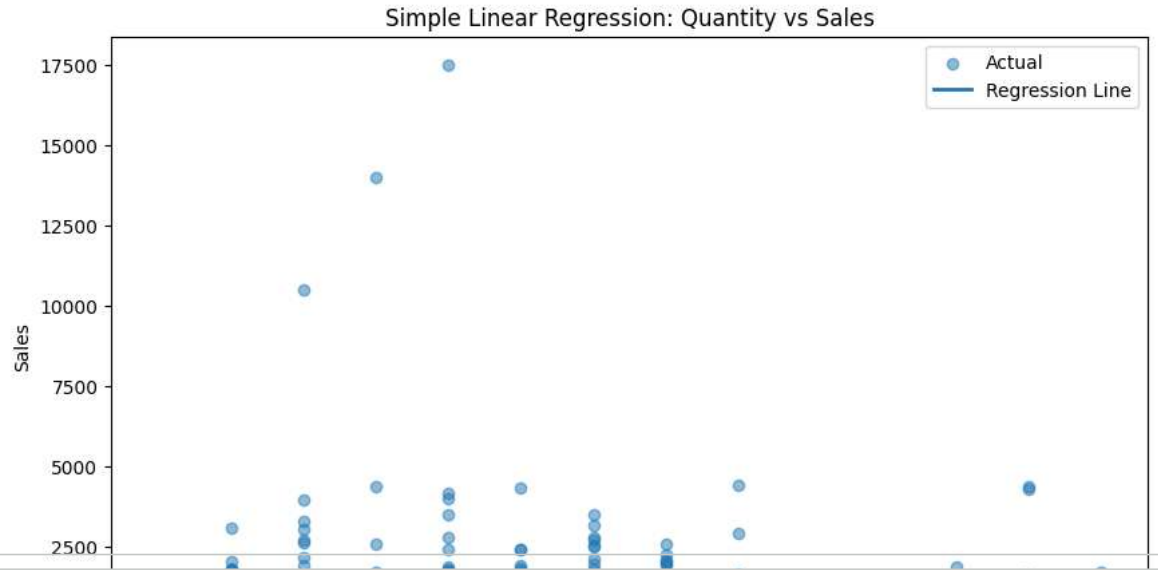



```
plt.figure(figsize=(10, 6))

plt.scatter(
    X_test_simple['Quantity'],
    y_test_simple,
    alpha=0.5,
    label='Actual'
)

plt.plot(
    X_test_simple['Quantity'],
    simple_pred,
    linewidth=2,
    label='Regression Line'
)
```

```
plt.title('Simple Linear Regression: Quantity vs Sales')
plt.xlabel('Quantity')
plt.ylabel('Sales')
plt.legend()
plt.show()
```




```
# ============================================================
# MULTIPLE LINEAR REGRESSION
# ============================================================

regression_features = [
    'Quantity',
    'Discount',
    'Category',
    'Sub-Category',
    'Region',
    'Segment'
]

X_multi = df[regression_features].copy()
y_multi = df['Sales'].copy()

print("Features used:")
print(regression_features)

print("\nDataset shape:", X_multi.shape)
```

```
Features used:
['Quantity', 'Discount', 'Category', 'Sub-Category', 'Region', 'Segment']

Dataset shape: (10194, 6)
```

```
# One-hot encoding

X_multi_encoded = pd.get_dummies(
    X_multi,
    columns=[
        'Category',
        'Sub-Category',
        'Region',
        'Segment'
    ],
    drop_first=True
)

print("Encoded feature count:", X_multi_encoded.shape[1])

display(X_multi_encoded.head())
```

Encoded feature count: 25

	Quantity	Discount	Category_Office Supplies	Category_Technology	Sub-Category_Appliances	Sub-Category_Art	Sub-Category_Binders	Category_B
0	2	0.2	True	False	False	False	False	
1	2	0.8	True	False	False	False	True	
2	3	0.2	True	False	False	False	False	
3	3	0.2	True	False	False	False	False	
4	3	0.2	True	False	False	True	False	

```
# Train-test split

X_train_multi, X_test_multi, y_train_multi, y_test_multi = train_test_split(
    X_multi_encoded,
    y_multi,
    test_size=0.20,
    random_state=42
)

print("Training samples:", len(X_train_multi))
print("Testing samples :", len(X_test_multi))
```

```
Training samples: 8155
Testing samples : 2039
```

```
# Train Multiple Linear Regression

multi_model = LinearRegression()

multi_model.fit(
    X_train_multi,
    y_train_multi
)

multi_pred = multi_model.predict(
    X_test_multi
)

print("Multiple Linear Regression model trained successfully.")
```

```
Multiple Linear Regression model trained successfully.
```

```
# ============================================================
# MULTIPLE REGRESSION EVALUATION
# ============================================================

multi_mae = mean_absolute_error(
    y_test_multi,
    multi_pred
)

multi_mse = mean_squared_error(
    y_test_multi,
    multi_pred
)

multi_rmse = np.sqrt(multi_mse)

multi_r2 = r2_score(
    y_test_multi,
    multi_pred
)

print("Multiple Linear Regression Results")
print("-" * 50)
print(f"MAE  : {multi_mae:.4f}")
print(f"MSE  : {multi_mse:.4f}")
print(f"RMSE : {multi_rmse:.4f}")
print(f"R²   : {multi_r2:.4f}")
```

```
Multiple Linear Regression Results
--------------------------------------------------
MAE  : 223.6580
MSE  : 402935.8762
RMSE : 634.7723
R²   : 0.2281
```

```
# ============================================================
# REGRESSION MODEL COMPARISON
# ============================================================

regression_comparison = pd.DataFrame({
    'Model': [
        'Simple Linear Regression',
        'Multiple Linear Regression'
    ],
    'MAE': [
        simple_mae,
        multi_mae
    ],
    'MSE': [
        simple_mse,
        multi_mse
    ],
    'RMSE': [
        simple_rmse,
        multi_rmse
    ],
    'R²': [
        simple_r2,
        multi_r2
    ]
})

display(
    regression_comparison.round(4)
)
```

	Model	MAE	MSE	RMSE	R²
0	Simple Linear Regression	266.3376	504020.3594	709.9439	0.0345
1	Multiple Linear Regression	223.6580	402935.8762	634.7723	0.2281

```
# ============================================================
# ACTUAL VS PREDICTED SALES
# ============================================================

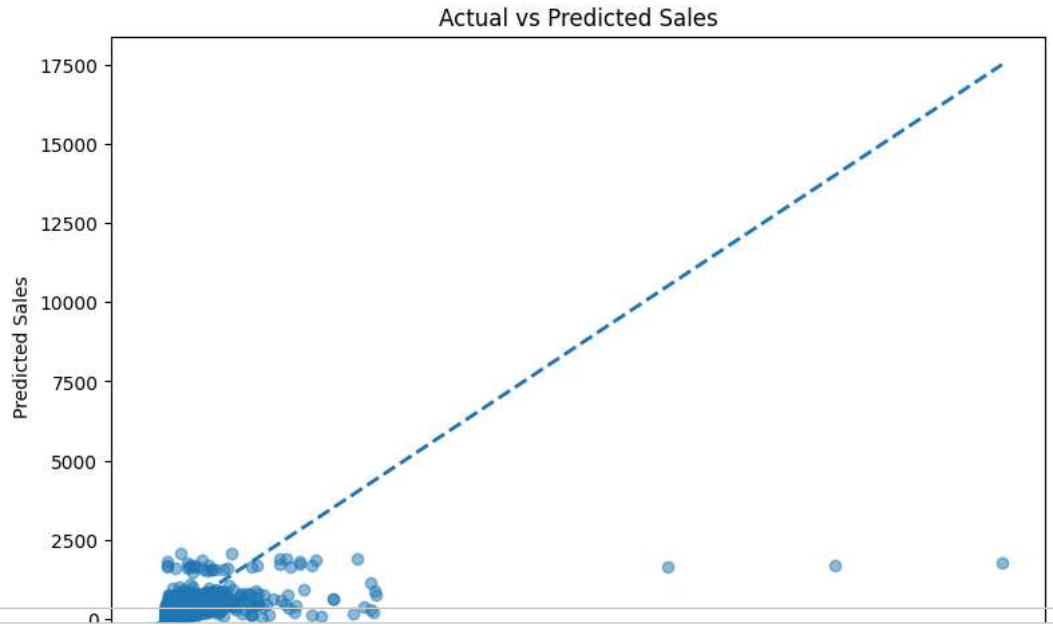
plt.figure(figsize=(9, 6))

plt.scatter(
    y_test_multi,
    multi_pred,
    alpha=0.5
)

# Perfect prediction line
min_value = min(y_test_multi.min(), multi_pred.min())
max_value = max(y_test_multi.max(), multi_pred.max())

plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    linestyle='--',
    linewidth=2
)

plt.title('Actual vs Predicted Sales')
plt.xlabel('Actual Sales')
plt.ylabel('Predicted Sales')
plt.show()
```




```
# ============================================================
# RESIDUAL ANALYSIS
# ============================================================

residuals = y_test_multi - multi_pred

print("Residual Statistics")
print("-" * 50)
print(f"Mean Residual : {residuals.mean():.4f}")
print(f"Std Residual  : {residuals.std():.4f}")
print(f"Minimum       : {residuals.min():.4f}")
print(f"Maximum       : {residuals.max():.4f}")
```

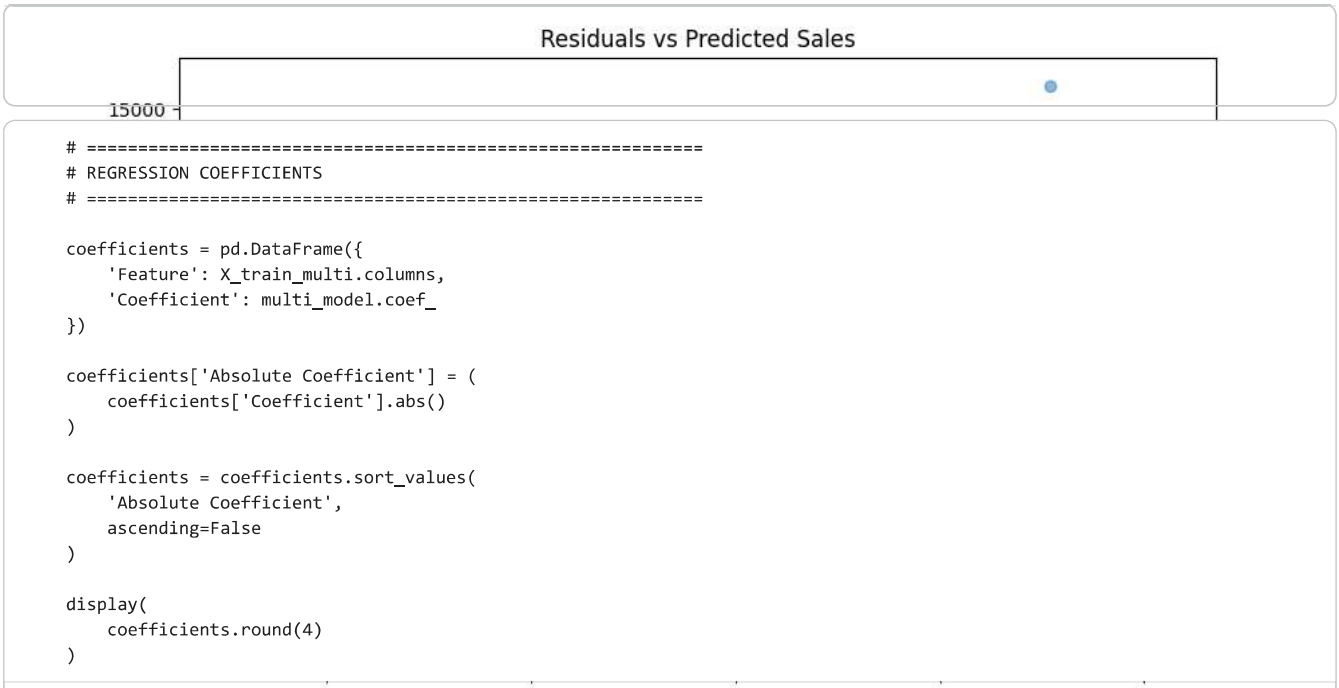
```
Residual Statistics
--------------------------------------------------
Mean Residual : 20.6738
Std Residual  : 634.5912
Minimum       : -1777.3201
Maximum       : 15730.7035
```

```
plt.figure(figsize=(10, 6))

plt.scatter(
    multi_pred,
    residuals,
    alpha=0.5
)

plt.axhline(
    y=0,
    linestyle='--',
    linewidth=2
)

plt.title('Residuals vs Predicted Sales')
plt.xlabel('Predicted Sales')
plt.ylabel('Residuals')
plt.show()
```




```
# ============================================================
# REGRESSION COEFFICIENTS
# ============================================================

coefficients = pd.DataFrame({
    'Feature': X_train_multi.columns,
    'Coefficient': multi_model.coef_
})

coefficients['Absolute Coefficient'] = (
    coefficients['Coefficient'].abs()
)

coefficients = coefficients.sort_values(
    'Absolute Coefficient',
    ascending=False
)

display(
    coefficients.round(4)
)
```

	Feature	Coefficient	Absolute Coefficient
14	Sub-Category_Machines	1580.3052	1580.3052
9	Sub-Category_Copiers	1459.2202	1459.2202
19	Sub-Category_Tables	327.1344	327.1344
1	Discount	-275.9050	275.9050
12	Sub-Category_Furnishings	-269.7615	269.7615
2	Category_Office Supplies	-226.9525	226.9525
7	Sub-Category_Bookcases	180.3595	180.3595
16	Sub-Category_Phones	175.5452	175.5452
3	Category_Technology	-167.0062	167.0062
8	Sub-Category_Chairs	156.2262	156.2262
11	Sub-Category_Fasteners	-141.4734	141.4734
13	Sub-Category_Labels	-131.4474	131.4474